Technical companion

This reference accompanies *From SFC Circulars to Reviewable Bank Controls*. The monograph explains the decisions, mathematics and limits of the control method. This companion begins with the implemented investigation through the round-four checkpoint, then retains provision inventories, field definitions, software interfaces, historical implementation records and commands for people reproducing or auditing the work. It uses public sources and synthetic cases. Its execution records do not establish independent legal approval or readiness for bank production.

The two PDFs are intended to be kept together. References between them link the reader's explanation to its reproduction record. All mathematical derivations needed for the main argument remain in the monograph.

Chak Wong 28 September 2026

Contents

Technical companion **i**

A Reproducing the implemented investigation **1**

A.1 Which route is implemented 1

A.2 Resolve an authority edition 2

A.3 Compose separate controls 2

A.4 Run a case that distinguishes readings 3

A.5 Use an official example without inventing authority 3

A.6 Present proposals and preserve partial checks 4

A.7 Freeze and execute a comparison 5

A.8 Package Java and revisit source changes 5

A.9 Find the retained evidence 7

B Additional prospectuses and jurisdiction cases **8**

B.1 Twenty issue-specific documents 8

B.2 Four programme documents 10

B.3 How to reproduce and interpret this record 11

B.4 Additional historical source dossiers 12

B.5 Thirty additional corner-case questions 14

C Technical implementation record **19**

C.1 Complete SPI provision inventory 19

C.2 Detailed inputs for the demonstrated Java profile 21

C.3 Full distinguishing-case table 23

D Historical implementation and design record **25**

D.1 The shared workbench and its users 25

D.2 What a reviewer should see 26

D.3 The execution architecture and its deliberate limits 26

D.4 The original work packages and their execution 28

D.5 The implemented authoring service and its current boundary 32

D.6 What the 22 September local execution established . . . 35
D.7 The scripted interpretation increment completed on 23 September . . . 43
D.8 Extend the existing product at a defined boundary . . . 44
D.9 Separate durable records from generated prose . . . 45
D.10 The run and policy records . . . 46
D.11 The candidate record carries the interpretive commitment . . . 47
D.12 The issue and action records make automatic work accountable . . . 48
D.13 Coverage records connect source units to consequences . . . 49
D.14 Storage transactions and event history . . . 50
D.15 An API with reviewable state transitions . . . 51
D.16 The review interface should expose the decisive difference . . . 53
D.17 Provider adapters are deliberately narrow . . . 54
D.18 Security protects the meaning of the evidence . . . 55
D.19 The ordered implementation sequence . . . 56
D.20 Acceptance scenarios that exercise the whole cycle . . . 58
D.21 The first vertical slice and its completion rule . . . 59

E Detailed comparison tables **60**
E.1 Ensemble responsibilities . . . 60
E.2 Resolution evidence . . . 60
E.3 Software and standards considered . . . 62
E.4 Cross-circular acceptance cases . . . 64
E.5 Evaluation evidence contract . . . 65
E.6 Operational readiness evidence . . . 66
E.7 Recording a method-comparison decision . . . 67

F Software interfaces and reproduction notes **68**
F.1 Catala shared frontend and reproduction boundary . . . 68
F.2 The earlier SPI compiler: a concrete deployment lesson . . . 70
F.3 The actual generated Java interface . . . 75
F.4 What can be reused from the adjacent projects . . . 76
F.5 Exact controller sequence . . . 79
F.6 Machine-readable blocked-result example . . . 80
F.7 Exact financial-rule serialization . . . 81
F.8 Complete rule-language grammar . . . 81

F.9 Durable request accounting and worker recovery 82
F.10 Release identity and reproducible builds 84

Appendix A

Reproducing the implemented investigation

This chapter records the interpretation and integration capabilities accepted through round four on 24 September 2026. It complements the explanations in the main volume with exact entry points and retained evidence. Earlier scripted increments are historical records in Appendix D. Later implementation work has its own dated reports; the counts here belong to this checkpoint and must not be read as a live resource balance.

A.1 Which route is implemented

The service can generate isolated readings, investigate alternatives with bounded breadth-first or UCT search, check independently collected source claims, preserve unsupported proposals and construct draft Java controls. The source inventories are collected separately from candidate generation; independence of requests does not establish statistical independence of errors. Composition combines explicitly assigned component controls. Executed scenarios can rank questions when their facts and outcomes are comparable. Registered official examples can supply conditional arguments. A local scheduler revisits affected controls after source changes.

The evidence addresses three targets: retained English and applicability, proposed interpretation and formal expression, and generated Java behaviour. Exact bytes and quotes support the first target's identity. Model judgments and source comparisons inform the second. Java replay checks the third within the exercised domain. No completion flag promotes one target into another. The full scope statement is retained at `docs/implementation/interpretation-round4/method-boundaries.md`.

Rounds two and three contain live development work. The accepted round-three investigations retained 90 claim/candidate results for 23EC46 and 384 for 24EC16. Of the latter, 224 are deterministic unavailable-program findings for seven unencodable proposals. They are not model judgments about legal entailment. The other 160 received source comparisons. All 474 pairs were accounted for, with both investigations unresolved and ineligible for release. Additional concerns and residual questions include programming, source and factual gaps; their counts are not counts of independent legal ambiguities.

Round four revalidated this evidence and used controlled or retained inputs, with no new model calls. Its final regression record contains 503 tests, zero failures, errors or skips, and two existing dependency warnings. The source-update exercise also ran a

separate Java host. Suite counts from successive phases overlap; summing them would misdescribe the evidence.

A.2 Resolve an authority edition

The catalog at `.localresources/sfc-authorities/b1/catalog.json` links an issuing authority to editions, provision locations and supported aliases. A citation in a candidate must resolve to a retained edition and exact text, not merely a similar title. The catalog records publication and date evidence separately from the requested assessment time. The accepted exercise covered four official documents and six locator/date resolutions, including the September 2023 and January 2026 Code editions.

The FAQ's title and its later answer updates require special care. The retained page is not an independently archived 2010 edition. Its historical applicability remains conditional, and remote statutory definitions remain unacquired. The extraction checks compare two text extractions and preserve disagreement or image warnings. OCR is not implemented. Matching extraction results cannot establish that a scanned qualification has been read.

An assurance source manifest may name `authority_registry`, `example_registry` and `example_scope`; relative paths resolve against that manifest. The scope states issuer, jurisdiction and issue IDs. The public example `examples/interpretation-assurance/authority-and-examples.json` uses the retained catalog and example registry. It is a development input, not an institution's approved source register.

A.3 Compose separate controls

The component specification binds the source packet, inventoried claims and complete final candidate set. Each component states its selected claims, alternative candidate IDs, output meaning and assignment basis. The caller must justify why the members are rival readings of one control, or separate controls that belong together. The tool does not infer that legal grouping from successful compilation.

The command's required arguments are:

```
.venv/bin/python -m legalmath.cli assurance-compose \
  --investigation INVESTIGATION_DIRECTORY \
  --spec COMPONENT_SPECIFICATION.json \
  --jdk JDK_DIRECTORY --out NEW_OUTPUT_DIRECTORY
```

These capitalized arguments stand for operator-supplied paths. Use a fresh output directory. A retained real-input specification is `artifacts/interpretation/round4/B2a/attempt-01/composition/23ec46-spec.json`; its identities belong to the referenced historical investigation and cannot be reused after changing the source or candidate.

The output includes bundles, generated Java, JARs, cases, conformance results and a

manifest. Alternative selections form separate packages. Within each package, rules and facts use component namespaces so unrelated meanings cannot collide. Each result retains its source claims, modality, scope, exceptions, timing and upstream uncertainty. Missing claims, unsupported expressions, unassigned candidates and combinations deferred by the work limit remain explicitly incomplete. The package is a draft.

Seven builds were exercised: two controlled two-component selections and five retained gift-control alternatives. True, false, unknown, conflict and out-of-scope results were covered. These are not seven whole-circular implementations; complete coverage of 23EC46 and 24EC16 remains unproved.

A.4 Run a case that distinguishes readings

The question procedure begins with a source-bound candidate comparison and its factual witness. The record includes the source packet, candidate set, assessment time, base candidate, subject, output convention, assumptions and source quotes. Before replay, the implementation checks subject and output compatibility and constructs the declared fact correspondence. The original candidate programs then run in Python and Java on the corresponding snapshots.

The partition record retains accepted replays, excluded candidates and reasons, work consumed and separated pairs. Unit-weight scores count executed pairs with different projected results. They are scheduling observations, not legal probabilities. Resource-deferred work cannot be labelled complete; unsupported programs and incompatible facts cannot earn a score. Source-inventory omission checks retain priority even if every comparable output agrees.

The controlled gift/discount case separated one pair and reached the repair scheduler. The retained 23EC46 comparison had incompatible fact bindings; 24EC16 had no checked pair. Both reported `NO_CHECKED_DISTINCTIONS`. This means no checked distinction was available, not that all possible readings agreed. Conditional derived comparisons remain a separate facility. The implemented partition and verifier are in `src/legalmath/interpretation/assurance/questions.py`; retained execution is indexed by `docs/implementation/interpretation-round4/b2b-result.md`.

A.5 Use an official example without inventing authority

The register at `.localresources/sfc-examples/b3/registry.json` binds each example to the authority catalog, exact source quotation, issuer, edition, proposed factors and declared outcome. The current issue is `paragraph.3.11.applicability`. It concerns whether the paragraph applies, not an unconditional prohibition after exceptions.

Two proposed branches of FAQ Question 1 produced twenty conditional analogy and distinction moves on five candidates. They share one source family. Six questions remained, including historical applicability and whether the selected factors were

legally material. The factorization is a developer proposal that requires review; genuine quoted words do not make it exhaustive.

A generated example outside the register remains an unregistered hypothetical proposal. A quotation cannot authenticate an invented court, case identity or holding. This implementation has no court-case corpus, general judicial-ratio extractor or procedure for determining binding priority. Source or registry changes invalidate dependent analysis instead of silently retaining its former applicability.

A.6 Present proposals and preserve partial checks

Present candidate analysis through `candidate-presentation.json`. An entry with `WITHHELD_CORRUPT_PROSE` has a null presentation and points to `prose-quality.json` and the original candidate. Raw proposals, compiled programs and their source checks remain retained. An entry with `NO_PATTERN_DETECTED` passes only the implemented pattern diagnostic. It has no certificate of natural language coherence or legal correctness.

The detector identified five defects in the actual retained 24EC16 proposal: placeholder subject and statement fields, self-repair chatter, and output instructions embedded in questions. Exact authentic quotations, ordinary uncertainty and non-English text are preserved by the checked negative cases. The detector's coverage is deliberately narrow. Editing the proposal would require a successor candidate and renewed checks; the presentation mechanism does not silently rewrite its meaning.

Fidelity results are checked for exact claim/candidate pairs, source quotations and representation quotations. The historical 23EC46 response contained 33 rows where 32 were required; the validator identified the extra pair, and the retained correction validated all 32. That was revalidation of an existing correction, not a new live repair.

The per-response schema permits at most 1,000 checks and 30 concerns. The aggregate profile bounds an investigation at 4,096 pairs, 1,080 concerns and 2,097,152 bytes; its per-call pair default is 32 with a maximum of 64. These are engineering capacity choices. Admission considers cached and unsupported readings as well as pending model work. It can conservatively refuse work that might have returned shorter responses.

A completed matrix is retained in `complete.json`. Interrupted aggregation keeps `partial.json` and the validated parts; an oversized cached part remains separately available. Repeated concerns are retained even when a separate presentation list deduplicates residual questions. A later failure cannot erase earlier rows or turn a partial result into a clean interpretation. The full profile and record names are in `docs/implementation/interpretation-round4/operator-guide.md`.

A.7 Freeze and execute a comparison

The implemented adapters are `single-reader`, `isolated-readers`, `search` and `assurance`. Freeze the source, assessment time, case-family assignments, reference outcomes and resource ceilings through `legalmath.interpretation.assurance.evaluation.freeze` before dispatch. The hidden scoring references are unavailable to the methods. Missing answers and abstention are reported alongside unsafe and useful accepted answers. The unit of uncertainty is a paired source-family average, as derived in Section 9.9.1 of the monograph.

A replay invocation has this form:

```
.venv/bin/python -m legalmath.cli assurance-evaluate \
  --frozen FROZEN_STUDY.json --out NEW_OUTPUT_DIRECTORY \
  --jdk JDK_DIRECTORY --replay-responses RESPONSES.json
```

For authorized live work, the provider arguments are instead `--allowance LEDGER.json --total-calls AUTHORIZED_CEILING`. The command checks the complete reservation before dispatch. Supplying the flag does not increase the authorized ledger ceiling.

The round-four fixture reserved eight calls per method for two cases, or 64 calls. At that checkpoint only 22 of the original 100 remained. Consequently no live comparison was run. The eight scripted jobs tested actual adapters, hidden finite-scenario scoring and adverse outcomes. Their one development family supplied no informative statistical separation and no estimate of English error or reviewer savings. Additional calls alone would not make those cases a held-out sample.

Subsequent work on references and source families is recorded separately in `docs/implementation/interpretation-round5/execution-report.md`. Those reference preparations and new-source attempts form a separate continuation. Historical call balances above must not be copied into a new run; use the actual ledger and that run's reviewed study. A complete effectiveness study still needs defensible outcome evidence and a justified sampling and precision design. Internal qualified reviewers or explicit official outcomes can support suitable tasks; there is no automatic external-consultancy requirement.

A.8 Package Java and revisit source changes

The package builder retains the draft rule bundle, policy source, JAR, host caller, source commitments, conformance cases and manifest. A separate Java process calls the generated public API and receives the full typed result. Unknown or conflict must not be reduced to a Boolean authorization. The local exercise uses draft mode and supplies no institution release identity.

The scheduler wraps the fixed assurance monitor with bounded visits and process timeouts. Its command accepts:

```
.venv/bin/python -m legalmath.cli assurance-schedule \
  --config REGISTRY.json --resource-root RESOURCE_DIRECTORY \
  --out NEW_OUTPUT_DIRECTORY --jdk JDK_DIRECTORY \
  --at ASSESSMENT_TIME --settings SETTINGS.json \
  --replay-responses RESPONSES.json \
  --ticks 1 --interval-seconds 60 --tick-timeout-seconds 600
```

The numbers shown illustrate a bounded replay invocation, not a recommended bank cadence. The default is one visit. The assessment time stays fixed during an invocation; the institution scheduler must supply the intended assessment time for subsequent invocations. Authorized live provider arguments use the same allowance and total-call pair as evaluation.

Before dispatch, the wrapper copies source, registry, settings and replay bytes into the output record and binds their identities. Editing an input file while a visit runs cannot change its frozen inputs. Failed or uncertain work consumes an attempt and remains visible. Nested failure or unresolved meaning produces an attention state even when the outer process exits successfully.

The final integration repair prevents an unperformed example query, a resource-deferred partition or limited argument analysis from being cached as fully executed. Completed but unresolved interpretation remains unresolved on unchanged visits. Retries are bounded, and corrupted cached evidence is rejected. The status distinguishes what ran from what was established.

The accepted update exercise launched three separate Java host processes. Adding a fee-discount exception changed the selected restriction from true to false on the controlled facts. An unaffected control and the earlier package were preserved. Corrupted packages, uncertain callbacks, missed cadence and retry exhaustion were also exercised. No actual regulatory amendment was independently interpreted in this demonstration.

For institution staging, the bank must supply its host and JDK, service identity, control register and fact bindings, operations-owned scheduler, update cadence and release roles. The local command installs no service and activates no bank policy. Confidentiality, authenticated release, audit integrity and concurrent host transactions require evidence from that environment.

A.9 Find the retained evidence

The acceptance state is in `artifacts/interpretation/round4/state.json`. Each accepted phase points to an immutable attempt directory and its `run-manifest.json`; inspect the manifest for exact commands, inputs, logs and output identities. The compact index below explains what each record establishes.

B0: proposal and matrix integrity
Actual corrupt prose and the extra historical response pair, plus capacity and partial-result retention. See `docs/implementation/interpretation-round4/execution-report.md`.

B1: source editions
Four documents and six locator/date resolutions, with missing definitions and historical uncertainty retained. See `docs/implementation/interpretation-round4/b1-result.md`.

B2a and B2b: composition and distinguishing cases
Seven Java builds and the controlled executed partition, with no comparable witness claimed for either retained circular. See the corresponding result notes in `docs/implementation/interpretation-round4/`.

B3 and B4: examples and evaluation
One official-example family and eight scripted comparison jobs. These establish conditional argument construction and evaluation mechanics. Their result notes state the remaining semantic and empirical limits.

B5 and B6: integration and completion
Local Java-host update and final status/input-retention repairs. The final suite is `artifacts/interpretation/round4/B6/attempt-01/tests.xml`; its adjacent manifest binds the 503-test acceptance and operations exercise.

The preceding live development is documented in the round-two and round-three execution reports under `docs/implementation/`. These records explain how proposals and source judgments were obtained; round-four replay does not turn them into fresh independent observations. The two retained circulars remain unresolved. No method ranking, population interpretation accuracy or reduction in review cost is established by this evidence index.

Appendix B

Additional prospectuses and jurisdiction cases

This appendix preserves the dated campaign results. The 6 October correction, literature review and solution appear in Section 2.6. The phase command is `python3 -m scripts.prospectus_delivery run`. Current repair receipts and refreshed work orders are under `docs/implementation/prospectus-repair-2026-10-06/`. The earlier successor directory preserves the historical 6 October execution. Campaign completion, legibility and independent legal acceptance are separate. Section 2.6.5 reports the executed results: P0–P6 remain partial, P7 has no independent evaluation and P8 blocks release. The ordered repair connects the saved construction, law and financial products to the installed report, exercises 25 retained law scenarios and three financial profiles, and preserves the remaining construction and review obligations. The saved-run verification command is `python3 -m scripts.verify_prospectus_repairs`; its test results and source bindings are in the repair directory's `finalization/` subdirectory.

This appendix identifies the sources used in the 4 October 2026 difficulty study and the subsequent jurisdiction-source investigation. The main argument is in Section 2.5. The document list serves exact lookup: each PDF link opens a retained local copy, and each nine-digit identifier matches the catalogue and raw reader result. Source page references elsewhere are physical PDF page numbers.

B.1 Twenty issue-specific documents

The frozen reader returned 19 abstentions and one conditional positive reading on these 20 documents, with 187 unresolved clause records. Multiple series in one prospectus count once. Ecuador's May offering and Volcan's 2026 offering are taps of their respective earlier series, not independent new instrument families. Every complete issue-level answer remains unadjudicated; *Yes** reports only the bounded reader's result.

Document and retained source	Reading	Observed difficulty
Prudentia S.à r.l., AirBlu compartment. €35m 4.50% notes due 8 June 2029. 2022-10-13. PDF 103216017; 109 pp.	Abstain; 3 unresolved	Unpaid claims extinguished after compartment assets are distributed; principal-loss boundary unresolved.
Hannover Rück SE. €750m 5.875% subordinated notes due 2043. 2022-11-10. PDF 103254064; 107 pp.	Abstain; 3 unresolved	German controlling terms, interest deferral and draft insurance-resolution law.

Document and retained source	Reading	Observed difficulty
Hannover Rück SE. €750m 1.375% subordinated notes due 2042. 2021-03-18. PDF 102149829; 102 pp.	Abstain; 2 unresolved	German controlling terms, deferral and amendment provisions.
SCOR SE. €500m 4.510% subordinated notes due June 2056. 2026-06-02. PDF 105421783; 90 pp.	Abstain; 14 unresolved	Pending national implementation of insurance-resolution powers; separate future law from current terms.
SCOR SE. €500m 4.522% subordinated notes due September 2055. 2025-09-05. PDF 104941226; 81 pp.	Abstain; 15 unresolved	Insurance deferral and resolution wording with date-specific applicability unresolved.
The Ethniki Hellenic General Insurance Company S.A. €100m 6.875% RT1 perpetual temporary write-down notes. 2026-09-07. PDF 105585556; 152 pp.	Yes*; 57 unresolved	Defined temporary principal write-down recognised; surrounding clauses remain unresolved.
The Ethniki Hellenic General Insurance Company S.A. €100m 5.250% Tier 2 notes due 2036. 2026-09-07. PDF 105585564; 129 pp.	Abstain; 14 unresolved	Greek IRRD implementation pending; existing powers and future powers must be separated.
Zurich Finance (Ireland) DAC. JPY50bn 2.979% senior notes due 8 July 2033, Series 70 tranche 1. 2026-07-08. PDF 105489584; 8 pp.	Abstain; 0 unresolved	Short supplement depends on the May 2026 base; debt and repayment table fields unresolved.
Orange S.A. US$6bn notes due 2029, 2031, 2033, 2036 and 2056. 2026-01-13. PDF 105155475; 95 pp.	Abstain; 2 unresolved	Court restructuring and cross-class cram-down require a separate predicate interpretation.
OMV Aktiengesellschaft. €750m 4.3702% perpetual subordinated NC5.5 notes. 2025-06-26. PDF 104827261; 170 pp.	Abstain; 0 unresolved	No accepted repayment witness despite zero unresolved candidates; perpetual/German terms.
OMV Aktiengesellschaft. €750m 2.500% NC6 and €500m 2.875% NC9 perpetual subordinated notes. 2020-08-27. PDF 101817538; 208 pp.	Abstain; 0 unresolved	Two perpetual series; repayment and controlling-language review still needed.
Électricité de France S.A. US$1.5bn 9.125% reset perpetual subordinated NC10 notes. 2023-06-15. PDF 103648885; 173 pp.	Abstain; 3 unresolved	Other OCEANE bonds contaminate scope; exclusion concerns exchanged/varied notes.
EDF S.A. US$2.05bn fixed-rate notes due 2029, 2034 and 2064. 2024-04-22. PDF 104127325; 136 pp.	Abstain; 5 unresolved	Other OCEANEs and replacement certificates for an unredeemed balance remain candidates.

Document and retained source	Reading	Observed difficulty
TenneT GmbH & Co. KG. €1.4bn 4.750% NC8.5 subordinated EuGB notes due March 2057, Series 6 tranche 1. 2026-09-01. PDF 105577212; 45 pp.	Abstain; 0 unresolved	Correct issuer is TenneT GmbH & Co. KG; repayment, language and base dependencies unresolved.
Republic of Ecuador. US$1bn tap of 8.750% 2034 and 9.250% 2039 notes. 2026-05-06. PDF 105373887; 252 pp.	Abstain; 9 unresolved	Tap of January series; historical consents and current amortization require separation.
Republic of Ecuador. US$4bn 8.750% 2034 and 9.250% 2039 amortizing notes. 2026-01-26. PDF 105190248; 237 pp.	Abstain; 8 unresolved	Two amortizing sovereign series; historical exchange and consent scope unresolved.
Hrvatska Poštanska Banka p.l.c. €150m 4.125% fixed-rate-reset callable senior preferred notes due 2029. 2026-07-09. PDF 105490784; 115 pp.	Abstain; 17 unresolved	Disclosed authority-ordered write-down of Notes not recognised by the reader.
Volcan Compañía Minera S.A.A.. US$220m tap of 8.500% senior secured notes due 2032. 2026-07-08. PDF 105488296; 425 pp.	Abstain; 17 unresolved	Tap of the 2025 series; subject binding in a long secured-debt memorandum.
Volcan Compañía Minera S.A.A.. US$750m 8.500% senior secured notes due 2032. 2025-11-14. PDF 105062309; 390 pp.	Abstain; 17 unresolved	Forfeiture of concessions/licences, equity changes and accounting liabilities contaminate scope.
AXA. €125m 3.47% senior notes due 17 June 2028. 2024-06-12. PDF 104224021; 65 pp.	Abstain; 1 unresolved	Proposed IRRD is conditional on adoption; explicit cash repayment wording was missed.

B.2 Four programme documents

The remaining four documents contain programmes or base terms. They produced two abstentions, two conditional positives and 69 unresolved clause records. A programme provision can support a clause-level finding without identifying a particular issued note. Mizuho's positive is also semantically wrong for its two paid-amortization witnesses. MUFG's positive has supporting text for the subordinated option.

Document and retained source	Reading	Observed difficulty
Zurich Insurance Company Ltd and programme financing issuers. US$18bn EMTN base prospectus. 2026-05-19. PDF 105392707; 371 pp.	Abstain; 20 unresolved	Programme issuer/guarantor selection and Swiss resolution wording unresolved.

Document and retained source	Reading	Observed difficulty
Volkswagen Financial Services AG and programme issuers. Three base prospectuses for VW financial-services programmes. 2026-09-02. PDF 105579250; 482 pp.	Abstain; 20 unresolved	Three programme prospectuses; unselected options cannot establish an issued-bond answer.
Mizuho Markets Cayman LP, guaranteed by Mizuho Bank Ltd. US$10bn medium-term note programme. 2026-08-31. PDF 105635287; 234 pp.	Yes*; 9 unresolved	Confirmed clause error: paid amortization classified as principal loss in unselected templates.
Mitsubishi UFJ Financial Group Inc. and MUFG Bank Ltd. US$50bn medium-term note programme base prospectus. 2026-08-07. PDF 105541740; 139 pp.	Yes*; 20 unresolved	Non-viability write-down recognised for subordinated option; no selected issue established.

B.3 How to reproduce and interpret this record

The protected run used commit `2d6b737b` plus 293 archived source files, including then-uncommitted repairs. It completed in 16.063 seconds on the existing CPU environment, with a 60-second guard per document and no classifier tuning after selection. Across all 24 documents there were 21 abstentions, three conditional positives and no runtime errors. The three focused harness checks and 909 prospectus regression tests passed; those checks establish software and preservation properties, not independent legal correctness.

The source set has 24 distinct hashes and 16 issuer families, below the original preference of at least 20 families. Orange had previous catalogue discovery exposure. The engine received whole-document diagnostic selections; that choice deliberately tests irrelevant text and unselected options, but does not establish their legal applicability. These distinctions prevent an abstention count from being presented as an error rate.

The convenient PDF index is `docs/prospectus/jurisdiction-2026-10-04/prior-24-pdf-index.json`. Each copy was checked against its retained original and SHA-256 hash. Original responses, HTTP receipts and extracted pages remain preserved; the new filenames do not replace them. The reviewed identities, dates, page anchors and raw-result links are in `docs/implementation/prospectus-difficulty-2026-10-04/reviewed-catalogue.json`. The same directory contains `review-errata.json` and `next-repairs.md`. The corrections include TenneT's issuer, EDF's replacement-note scope and proposed/future insurance-resolution law. Frozen pre-run metadata is retained so that the original run remains reproducible.

The immutable reader outputs and execution manifest are stored here:

```
docs/implementation/prospectus-difficulty-2026-10-04/run-001/
```

The method archive is `docs/prospectus/difficulty-2026-10-04/method.zip`. Integrity is checked with the following command from the worktree:

```
.venv/bin/python scripts/run_prospectus_evidence_master.py difficulty verify
```

That verifier reports the current cumulative request difference from the old freeze; subsequent jurisdiction retrievals must therefore be distinguished from the original study's 40 requests. The original study manifest and its dated verification preserve the earlier balance. The acquisition limit is a resource control, not a measure of legal completeness.

B.4 Additional historical source dossiers

The following documents are new to the retained source set. They are candidates or controls for external-law interpretation, not three verified examples of laws overriding a promise of repayment. Their immutable acquisition receipts remain in the shared request history, including the failed court downloads. No court decision is represented by an empty HTTP response.

B.4.1 BES: repayment promises and resolution measures

The retained final terms are dated 6 May 2014, Series 36, tranche 1, EUR 750 million of 2.625 percent senior notes due 8 May 2017. Rendered page 4 confirms ISIN PTBEQKOM0019 and Common Code 106485003. Rendered PDF pages 1–3 establish the issue, senior status, 100 percent redemption and EUR 100,000 final redemption amount per calculation amount. Page 5 says that intended Eurosystem eligibility does not establish actual eligibility: that depends on the Eurosystem criteria.

The terms require the 17 July 2013 base prospectus and supplements dated 2 September 2013, 6 November 2013, 24 February 2014 and 23 April 2014. The extended search retained that base and all four supplements. The Portuguese measure with its exact instrument list remains missing; the new judgments discussed below concern distinct claims that must be matched separately. The original five-page PDF has no extracted text layer; silent empty-text classification would lose even the repayment promise. Selected fields were transcribed from rendered pages, with their transcription status recorded.

The local source is `docs/prospectus/jurisdiction-2026-10-04/sources/jur-bes-2014-ptbeqkom0019/source.pdf`. A proposed test first establishes the exact promise, then adds a reviewed measure tied to that identifier and date. It must distinguish an authority's power, its exercise, a liability's transfer or retransfer, recognition in another forum, and the eventual payment outcome. A judgment concerning a different loan or note cannot fill the identifier link.

B.4.2 Greece: English-law principal protection and enforcement limits

The source is the Hellenic Republic's offering circular of 28 March 2008: EUR 600 million of 2.085 percent inflation-linked notes due 25 July 2057, consolidated with the EUR 1 billion 2007 issue. PDF page 25 gives permanent ISIN XS0292467775 and temporary ISIN XS0347909490. Condition 4, page 10, floors redemption at principal. Condition 10, pages 14–16, contains creditor-voting modification provisions. Condition 14, pages 17–18, selects English law and qualifies execution/attachment by international conventions and Greek law.

This is a useful control against silently applying a domestic-law statutory restructuring rule based only on the issuer country. It is also a positive source example of a contractual immunity waiver with an express local-enforcement qualification. The applicable law, actual attachment proceeding and exact restructuring history remain separate evidence needs. A readable official copy of *Kuhn*, C-308/17, is now retained. Its jurisdiction ruling concerns a statutory collective-action-clause exchange; it does not establish the treatment of these English-law Notes.

The source is `docs/prospectus/jurisdiction-2026-10-04/sources/jur-hellenic-2008-candidate/source.pdf`.

B.4.3 Slovenia: issuer identity and statutory guarantees

The 9 March 2011 information memorandum is issued by SID–Slovenska izvozna in razvojna banka, d.d., Ljubljana. It concerns a EUR 350 million tap of 3 percent notes due 21 April 2015, with an unconditional and irrevocable guarantee described on the cover. The permanent ISIN is XS0504013912 and temporary ISIN XS0605118966 (PDF p. 48). English law governs the Notes (p. 18); the statutory guarantee description refers to articles 11–13 of the Slovenian Export and Development Bank Act and a written creditor request (p. 24).

NLB appears in the document in other roles, which explains the catalogue match. It is not the issuer of these Notes. The source is retained as a negative control for an incorrect NLB/Kotnik attribution, and as a candidate for checking a statutory guarantee against its governing legislation. The extended search retained *Kotnik*, C-526/14. Its analysis of state-aid burden sharing does not establish a write-down or guarantee override for this SID issue.

The source is `docs/prospectus/jurisdiction-2026-10-04/sources/jur-nlb-2011-candidate/source.pdf`. The acquisition key preserves the failed search hypothesis; the reviewed catalogue records the corrected issuer.

B.4.4 The earlier acquisition checkpoint

The initial jurisdiction campaign ended at 112 cumulative public requests. Its failed court-source responses and three downloaded PDFs remain in `docs/prospectus/`

`jurisdiction-2026-10-04/`. The user then authorised 100 additional requests. The extension below used 58 of them, reaching 170 cumulative receipts and leaving 42 available. Historical reports retain their earlier counts; the current documents report the extended evidence.

B.5 Thirty additional corner-case questions

The extension retained 24 new PDFs: 17 offering or dependency documents, four UK judgments and three HETA notices. Six additional CJEU judgments were retained as readable official HTML. These are 30 substantive source documents, not 30 independent issuers, adverse events or engine tests. Discovery responses, redirects and host challenges remain in the request ledger and are excluded from that substantive count.

The reviewed catalogue and the test matrix contain the full source hashes, physical page or judgment paragraph, exact quotation, expected distinction, forbidden inference and unresolved evidence need. Scanned fields are stored as reviewed transcriptions, separately from the raw text supplied to the reader. All 51 recorded anchors verified. Independent legal adjudication remains pending.

B.5.1 Offering documents and incorporated terms

BES Series 23 final terms of 14 July 2011 identify EUR 81.4 million, 6.875 percent, maturity 15 July 2016 and ISIN PTBEQBOM0010. The six-page scan selects Portuguese law on page 3. Its base is dated 3 November 2010, with supplements of 15 November 2010, 3 March, 19 April, 6 June, 17 June and 13 July 2011. The bracketed date of the last supplement appears as such in the final terms. All seven named documents are now stored.

Series 35 final terms of 20 January 2014 identify EUR 750 million of 4 percent senior notes due 21 January 2019, ISIN PTBENKOM0012 (PDF p. 4). They require the July 2013 base and only its first two supplements. The previously retained Series 36 terms of May 2014 require all four supplements. These combinations prevent a newer programme edition or a later supplement from being attached to an older issuance without review. Incorporated financial reports, complete precedence analysis and the original Portuguese resolution annex remain open evidence needs.

New offering PDF	Retained local source and role
BES Series 23, 2011	Final terms; six-page scan. Par redemption and Portuguese law; candidate match to the coupon and maturity in C-500/22.
BES Series 35, 2014	Final terms; six-page scan. A different coupon, maturity and identifier.

New offering PDF	Retained local source and role
BES programme, 2010	Base prospectus; 288 pages. Senior and subordinated options, two issuers, guarantee and version-selection controls.
BES programme, 2013	Base prospectus; 231 pages. Supplies dependencies for the two retained 2014 final terms.
Adriano Lease Sec., 2011	Class A notes due 2041; 211 pages. Insufficient-funds cancellation with an improper-withholding exception; certificate and notice before exhaustion of claims (pp. 140, 144–145).
Adriano Finance, 2009	Class A RMBS due 2065; 278 pages. Article 67 claw-back exemption does not establish an Article 65 exemption for early payments; debtor categories matter (p. 65).
Adriatica Finance, 2007	EUR 140.95 million ABS due 2021; 227 pages. Regional health receivables, protected public funds, enforcement delay, unaccelerated underlying instalments and later cancellation (pp. 37, 43–46, 121).

The three acquisition keys beginning `hypo` preserve failed search hypotheses. Their issuers are the Italian companies printed in the table. HETA notices must not be attached to them.

B.5.2 Judgments, notices and the limits of their holdings

The official 2024 *Novo Banco* judgment, C-498/22 to C-500/22 records a bond retransfer and nonpayment (paras. 58–63), and addresses recognition of measures adopted before proceedings (paras. 89–97). The 2021 judgment, C-504/19 concerns a share-sale liability changed during pending proceedings (paras. 49, 59–66). *Goldman Sachs v Novo Banco*, [2018] UKSC 34 concerns the Oak loan and recognition of a determination that its liability had never transferred (paras. 21–34, PDF pp. 9–12). These are three distinct disputes.

In *Kotnik*, C-526/14, ECLI:EU:C:2016:570, the Banking Communication cannot impose independent obligations on Member States (para. 44). The burden-sharing discussion preserves qualifications, including the no-creditor-worse-off principle (para. 77), and distinguishes authority-adopted measures from private arrangements for the Directive 2001/24 classification (para. 110). *Kuhn*, C-308/17, ECLI:EU:C:2018:911, addresses the civil-and-commercial-matters jurisdiction boundary, not the merits of a particular English-law bond's haircut (para. 44).

Banco Santander, C-410/20, ECLI:EU:C:2022:351, addresses specified prospectus damages and nullity claims after a total *share* write-down (para. 51). Its discussion of

compensation compares resolution with normal insolvency; it does not guarantee the original investment's value (paras. 48–50). *Indėlių ir investicijų draudimas*, C-671/13, ECLI:EU:C:2015:418, makes negotiability material to deposit-certificate exclusion (paras. 38–40). Exclusion from deposit protection does not automatically exclude investor protection, whose own conditions still apply (paras. 47–52). In particular, article 2(2) of Directive 97/9 concerns the inability to return money or instruments held in connection with investment business. The judgment is not a general guarantee against an issuer's credit default.

The retained Dana Gas judgment, Lloyds ECN judgment and Ukraine judgment supply the controls developed in the main text: assumption versus finding, separate governing laws, regulatory redemption versus conversion, majority versus dissent, and permission to defend versus final discharge. Their original offering documents remain missing.

HETA's November 2014 rename notice, March 2015 maturity notice and April 2016 haircut notice are issuer disclosures. The last document is dated 11 April, despite a different catalogue date; it reports a 10 April decree. It describes eligible nonsubordinated amounts reduced to 46.02 percent, later interest reduced to zero, and a latest maturity of 31 December 2023. The original decree, exact bond and guarantee scope, subsequent modifications and ultimate recoveries remain unverified.

B.5.3 The test matrix

Each row denotes a distinct question. *Offering* means that a relevant original was included in the seven-document diagnostic; *clause* denotes one of the three additional probes. *Law* and *notice* are retained source specifications awaiting an integrated evaluator. None is presented as an independently adjudicated end-to-end legal pass.

ID	Source family	Distinction to preserve	Scope
CC01	BES scan	Empty text requires OCR; it cannot prove absence.	Offering
CC02	BES / Novo	Coupon and maturity match is provisional without the identifier annex; prior payment does not fix later liability.	Law
CC03	BES	Match the issue's base edition and supplement dates.	Offering
CC04	BES	Separate issuer and guarantor roles.	Offering
CC05	BES	Select senior terms before using programme options.	Offering
CC06	Novo 2024	Pre-suit recognition despite publication defect; preserve appeal access.	Law
CC07	Novo 2021	Protect pending proceedings against the specified retroactive retransfer.	Law
CC08	Oak loan	A loan holding does not establish the bond list.	Law
CC09	Oak loan	Challengeability does not itself suspend an act.	Law
CC10	HETA	A maturity stay is not principal cancellation.	Notice

ID	Source family	Distinction to preserve	Scope
CC11	HETA	Distinguish remaining percentage, haircut, interest dates and liability class.	Notice
CC12	HETA	Rename, related network and keyword search hits are different identity relations.	Notice
CC13	Dana Gas	Bind each obligation to its law and allocated risk.	Law
CC14	Dana Gas	An assumed fact is not a judicial finding.	Law
CC15	Lloyds	A regulatory call differs from conversion and write-down.	Law
CC16	Lloyds	Attribute majority and dissent separately.	Law
CC17	Ukraine	A triable defence is not debt discharge.	Law
CC18	Kotnik	Commission policy is not an independent national obligation.	Law
CC19	Kotnik	Private burden sharing differs from authority-adopted reorganisation.	Law
CC20	Kuhn	Jurisdiction classification does not decide haircut merits or another series' law.	Law
CC21	Popular	Share-specific remedy limits and insolvency comparison do not become bond-wide rules.	Law
CC22	Snoras	Negotiability conditions a deposit exclusion.	Law
CC23	Snoras	Deposit and investor schemes have distinct conditions.	Law
CC24	Adriatica	Public receivables and a seller undertaking do not guarantee the Notes.	Offering
CC25	Adriatica	Pari passu ranking does not make protected funds attachable.	Offering
CC26	Adriatica	Preserve the disclosed 120-day enforcement delay.	Offering
CC27	Adriatica	Note acceleration does not accelerate regional instalments.	Offering
CC28	Adriatica	Preserve later cancellation and the misconduct exception.	Clause
CC29	Adriano Lease	Preserve improper-withholding exception and certificate/notice conditions.	Clause
CC30	Adriano Finance	An Article 67 exemption does not imply an Article 65 exemption.	Offering

B.5.4 Frozen diagnostics and reproduced failures

All seven whole-document runs abstained. The two scanned final terms yielded no extracted clauses; the two BES bases left 17 and one unresolved records; the lease, RMBS and regional-health securitisations left two, five and four, respectively. The total is 29 unresolved records, with no runtime errors. These counts describe the bounded reader, not legal completeness or a failure rate in a representative population.

The three clause probes also abstained, but their internal classification discarded

unpaid-cancellation language as cancellation after redemption or repurchase. The independent regression screen checks the original passages and requires those clauses to remain recognised as unpaid loss or unresolved meaning. It fails on all three, as intended for these preserved defects. A passing integrity check therefore coexists with a failing interpretation screen. Exception-preserving interpretation would require more than this minimum screen.

From this worktree the retained results can be checked with:

```
python3 -m scripts.run_prospectus_corner_tests verify
python3 -m scripts.check_prospectus_corner_findings
```

The first command verifies hashes and the recorded run. The second exits with status 1 while the three known clause defects remain. Neither command makes network requests or modifies the frozen reader.

Sources are under `docs/prospectus/corner-cases-2026-10-04/`. The catalogue, 30 specifications, regression findings, raw outputs, build manifest and reset memo are under `docs/implementation/prospectus-corner-cases-2026-10-04/`. The earlier 24-document study and its source selections remain unchanged.

Appendix C

Technical implementation record

The main chapters use a small number of examples to teach the decisions. This appendix preserves the exhaustive mappings that an implementation or audit may need to look up. The tables are reference material; they are not intended to carry the argument without the surrounding explanations in Chapter 2.

C.1 Complete SPI provision inventory

Table C.1: Disposition of the complete circular for the worked profile. Subparagraphs within each stated range remain part of that row.

Source	Substantive instruction	Treatment in the worked change
Main text; Annex 1 opening and footnote 1	Introduce streamlining, retain conduct and risk controls, and import definitions and suitability requirements.	Record authority, perimeter and dependencies. The Java result covers selected conditions only; the bank retains its wider control chain.
1–2	Individual criteria at the relevant date; separate investment-holding corporation route.	Individual PI assessment is an input. A corporate client returns outside profile; ownership and qualifying-owner aggregation require a separate rule family.
3.1(a)–(b)	Inclusive alternative wealth thresholds and currency equivalents.	Execute exact HK-cent comparisons joined by OR. Foreign currency requires an adopted conversion mapping before evaluation.
3.2(a)–(d), 3.3, 3.4(a)(i)–(iii), (b)	Portfolio and asset/liability attribution; residence exclusion; written or equal joint shares under the stated circumstances.	Reviewed normalization produces portfolio and net assets. A missing agreement is unresolved evidence, not established absence. Preserve each ownership calculation.
4.1(a)–(d), 5.1	Reasonable sophistication assessment, alternative supporting routes and non-conservative objectives.	Execute the alternative routes and require the dated assessment and objectives. Do not substitute the representative's attributes.

Source	Substantive instruction	Treatment in the worked change
6, 7.1–7.3	Restrict transactions to selected categories and threshold; define categories, provide information, and apply category qualification.	Require selected category, resolved category explanations and information. Count same-category transactions or establish another qualification route.
8.1–8.2	Client specifies a threshold appropriate to circumstances; bank retains the setting and rationale.	Accept an agreed, reviewed absolute HKD threshold and rationale. A percentage-of-AUM arrangement needs a versioned conversion to the same input.
8.3(a)	Monitor gross exposure on execution.	Compare projected exposure including the candidate transaction with the agreed limit; host must reserve exposure atomically before using the decision.
8.3(b)	Alternative monitoring for designated accounts and new funds.	Explicitly outside profile. Implement its own account, transferred-position and top-up events before enabling it.

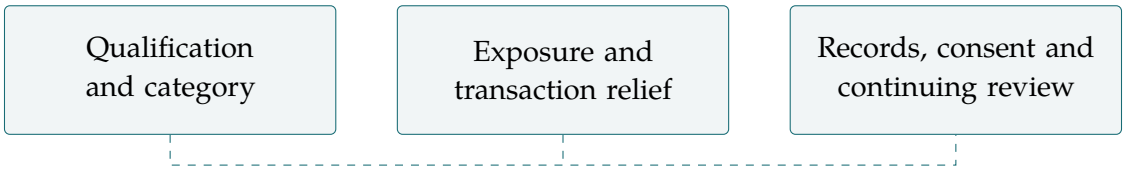


Figure C.1: The provision inventory preserves responsibilities beyond the financial test.

Source	Substantive instruction	Treatment in the worked change
8.4–8.5	Detect outsize or material transactions and warn; review threshold compliance annually and alert if exceeded.	Require transaction assessment and conditional warning. Schedule review/alert tasks; a reviewed composite reports annual controls current.
9; 10.1–10.4	Define solicited-transaction relief, with explanation exceptions and offering documents retained.	Execute the solicited profile. The result identifies applicable relief; it does not disable every suitability or disclosure control.
11.1–11.5; footnotes 2–4	Different relief for unsolicited complex-product transactions, including document alternatives and warning arrangements.	Outside profile. Preserve the distinct document/evidence branches, bond information alternatives and annual-warning requirement for a later implementation.

Source	Substantive instruction	Treatment in the worked change
12.1–12.3	Reasonable satisfaction, written assessment and correspondence recording category and threshold choices.	Dated assessments and document references feed named facts; records remain independently inspectable.
13.1(a)–(d), 13.2	Prior written agreement and explanation of consequences, categories, threshold and withdrawal/amendment rights.	A checklist-backed acknowledgment input plus a replayed consent state. Every item in the checklist must be evidenced; a signature alone is insufficient.
14.1–14.2	Annual continued-qualification/agreement review and written reminders covering consequences, categories, threshold, breaches and rights.	Ongoing tasks and reviewed status. Suspending streamlining when review is overdue is a proposed additional bank restriction.
Annex 2, FAQ 1	Appropriate reliance on disclosures after reasonable KYC efforts; clarify inconsistency.	Define admissible evidence and escalation; do not invent a blanket requirement for independent third-party confirmation.
FAQ 2	Client and power-of-attorney distinctions.	Attribute qualifications to the client. Apply the FAQ to transaction-history evidence without copying the attorney's sophistication.
FAQs 3–4, including notes	Category design, difficult product distinctions and category information.	Bank-owned category taxonomy, versioned mappings and information checklist; uncertain mapping requests review.
FAQ 5	Threshold should reflect the client's circumstances; bank-held AUM may be a limited part of wealth.	Recorded client circumstances and threshold rationale. No universal percentage is invented.
FAQ 6	Explain both monitoring approaches, including designated-account transfers, appreciation and new funds.	Use execution monitoring here. Keep alternative-account rules visible; do not infer forced unwinding merely from exceeding its threshold.
FAQ 7	Gross exposure includes leverage.	Feed leverage-inclusive projected exposure to the execution comparison.

C.2 Detailed inputs for the demonstrated Java profile

Table C.3: Input contracts for the demonstrated Java profile. A known value always has evidence, a validity interval and a recording time.

Input or group	Meaning and intended owner
Profile flags	`profile_individual`, `profile_solicited`, `profile_execution_monitoring`: the host's classified client, activity and monitoring mode. False selects an unsupported profile; unknown requests review.
`individual_pi`	Compliance assessment under the adopted Professional Investor Rules and Code definitions, for this legal client and date. It is distinct from merely being an individual.
`portfolio`; `net_assets_ex_home`	Valuation/operations supplies exact HK cents after reviewed ownership, liabilities, residence and currency treatment. Preserve the calculation inputs and source evidence.
Qualification fields	`relevant_degree`, `professional_qualification`, `relevant_year_of_work`; alternatively `category_transactions_3y`. Client-level evidence is mapped to the selected category and relevant period.
Assessment fields	`reasonable_sophistication_assessment`, `non_conservative_objectives`, `written_assessment_retained`. Compliance/relationship management supplies the assessment and records; missing review is unknown.
Category fields	`category_selected`, `category_information_delivered`, `category_explanation_resolved`. Client choice and delivery evidence must match the transaction's versioned category.

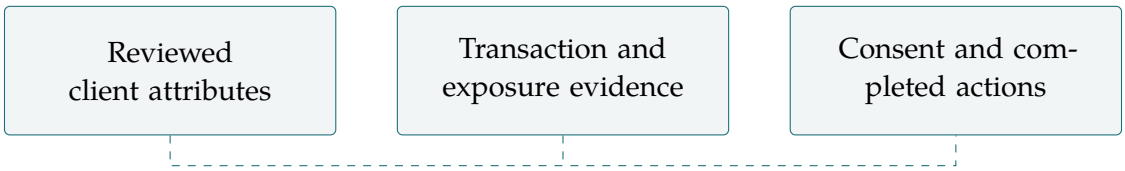


Figure C.2: The input table joins facts supplied by different operational owners.

Input or group	Meaning and intended owner
Acknowledgment and consent	`prior_written_acknowledgment_complete` covers every required item in 13.1–13.2; `active_consent` comes from the client/category consent history. Document completeness and present consent are different facts.

Input or group	Meaning and intended owner
Threshold and exposure	`streamlining_threshold`, `projected_gross_exposure` are HK cents; `threshold_rationale_retained` is an assessment/record check. Order management supplies a consistent exposure snapshot including leverage.
`annual_review_current`	Reviewed composite covering continued qualification/agreement, threshold review, required reminders and breach alerts. The demo consumes this status; it does not calculate annual deadlines or send reminders.
Product information	`current_offering_documents_delivered`, `explanation_requested_or_material_query`, `explanation_delivered`. Delivery/interaction records determine whether the remaining request has been answered.
Transaction warnings	`transaction_materiality_assessed`, `outsize_or_material_transaction`, `material_warning_delivered`. The assessment is required; a warning is required if the assessment says so.

C.3 Full distinguishing-case table

Table C.5: Distinguishing cases derived from the running interpretation. Other inputs remain as in the successful synthetic case unless stated.

Changed fact	Result	Why it changes the operational response
Portfolio HK$39,999,999.99; net assets HK$75m	Standard process	Both financial alternatives are refuted.
Portfolio HK$35m; net assets HK$90m	Conditions met	The alternative net-asset route suffices.
Portfolio missing; net assets HK$90m	Conditions met	Missing portfolio is reported, but does not block the established alternative.
Portfolio missing; net assets HK$70m	Review required	Portfolio evidence can change the answer. It is a blocking input.
Conservative objectives	Standard process	Wealth does not overcome paragraph 5.1.
Four category transactions; no other qualifying route	Standard process	The countable route is not established.
Chosen category does not match	Standard process	General SPI attributes do not extend the selected categories.

Changed fact	Result	Why it changes the operational response
Gross exposure HK$10,000,000.01	Standard process	The execution-monitoring condition is exceeded.
Written acknowledgment incomplete, or consent withdrawn	Standard process	Prior documentation and current consent are both required.
Client requests explanation; no answer recorded	Standard process	Paragraph 10.3 retains the requested explanation.
Material transaction; no warning delivered	Standard process	The applicable paragraph 8.4 warning remains outstanding.
Two inconsistent portfolio records	Review required	An evidence conflict is not resolved by choosing the more convenient amount.
Designated-account monitoring selected	Outside profile	Its different paragraph 8.3(b)/FAQ 6 logic has not been executed by this library.

Appendix D

Historical implementation and design record

This appendix preserves the original workbench and the 23 September scripted interpretation increment, together with their design proposals. Its task names, API sketches and future-tense passages belong to those checkpoints. For the implemented source generation, search, assurance, composition and monitoring through 24 September, begin with Appendix A. Empirical effectiveness and institution acceptance remain separate requirements.

D.1 The shared workbench and its users

LegalMath should be a regulatory specification workbench. Its primary users are the compliance officer interpreting a change, the product or operations owner deciding how it affects the business, and the engineer implementing the resulting control. An auditor or independent reviewer should be able to reconstruct the same decision later. The product succeeds when those people can inspect one maintained interpretation through views suited to their work.

The first local release was designed around a narrow end-to-end task: take a selected public circular and its dependencies, prepare proposed rules, resolve or retain interpretation questions, evaluate synthetic cases, and export a versioned change package containing generated Java source, a compiled JAR and its evidence. That local path is now implemented with public sources and synthetic identities. Integration into bank systems still follows the bank's ordinary change process.

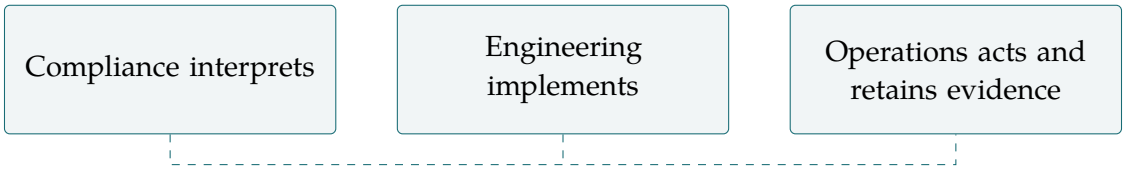


Figure D.1: The workbench gives different participants views of the same reviewed meaning.

D.2 What a reviewer should see

Opening the SPI financial condition should display the source passage and its annex context beside a controlled explanation: the portfolio route or the net-asset route can satisfy this particular condition. Definitions should be available in place, especially ownership attribution and exclusion of the primary residence. The same screen should show the boundary cases, missing evidence and where the condition is used in the larger SPI assessment.

The reviewer can change an interpretation, ask for a distinguishing example, or record a question for the policy owner. If two candidate readings differ only at the threshold, the workbench should produce that boundary case. If they differ because one applies to a representative and another to the client, the example should name the roles. The user should not need to understand a solver formula to see why the choice matters.

The engineer views the corresponding types, data mappings, executable dependency graph and tests. The operations owner views required actions, responsible teams, timing and evidence of completion. A diagram or decision table is generated from the same adopted rules. Maintaining a separate hand-edited flowchart would create another translation boundary and another version to reconcile.

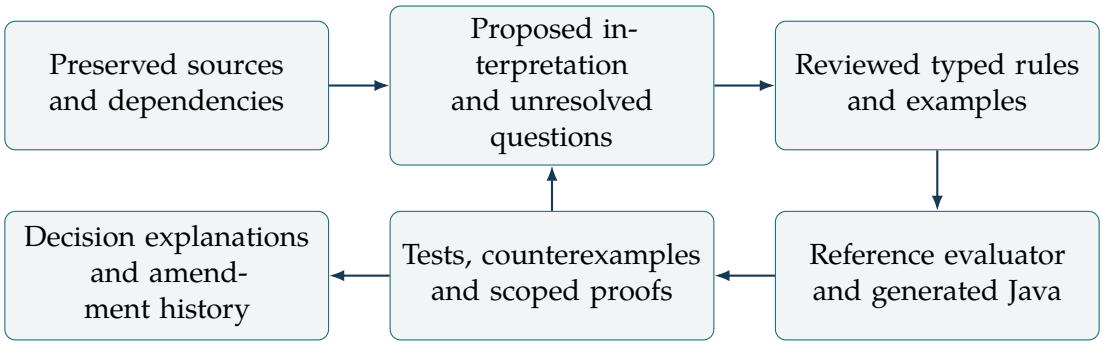


Figure D.2: The proposed workbench. Counterexamples return to interpretation review. A proof result remains attached to the exact rules, assumptions and software version it checks.

D.3 The execution architecture and its deliberate limits

The authoring runtime is a Python 3.11 reference interpreter, with JSON Schema and semantic validation, SQLite storage, content-addressed source files and a local FastAPI service. A server-rendered interface presents source, rule and case together. The interpreter walks a validated syntax tree; it does not execute model-generated Python. Sets, quantifiers, general recursion, arbitrary code, currency conversion and unbounded temporal formulas are outside RuleIR 0.1. An external, evidenced normalizer can supply a reviewed count or HKD value.

These restrictions keep common questions tractable and make the trusted code small enough to inspect. They also provide a natural place to evaluate the literature: Catala

for the decision fragment, DMN as a possible Java integration comparison, and a Stipula-inspired state model for a bounded workflow. The project should not integrate all candidate engines before demonstrating one complete use case.

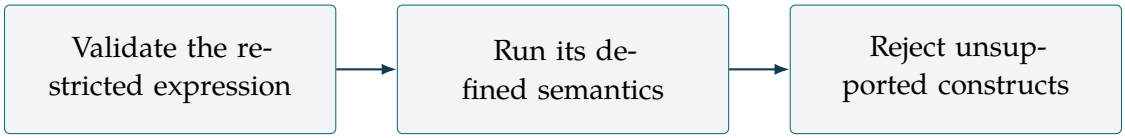


Figure D.3: A bounded language makes its supported behavior inspectable.

A reference interpreter implements the written execution semantics. A schema validator checks structure; a semantic validator checks expression types, resolved dependencies, source links and explicit exception relationships; a scenario runner evaluates reviewed examples. A solver adapter then searches for conflicts, unreachable branches and semantic differences. Generated code should come from a deterministic compiler for the supported fragment. At transaction time, evaluation uses an approved rule version and a recorded fact snapshot; language-model generation belongs to authoring and review.

An implementation adapter is accepted only after its treatment of missing values, numeric boundaries, priority, dates and obligations agrees with the reference semantics on the required checks. DMN's hit-policy choices and Rego's undefined results make this comparison substantive (Object Management Group, 2024; Open Policy Agent contributors, 2026). A backend that cannot express a required distinction should report that limitation instead of approximating it silently.

D.3.1 Source identity, review and storage

The source store preserves original bytes and an exact text derivative. The raw document and derivative each have a SHA-256 digest. A source span records the derivative hash, page, half-open character offsets and a hash of the quoted text. Extraction changes create a new derivative; they do not silently move an approved span. A reviewer can open the original page beside the extracted passage. Scanned or poorly extracted clauses require visual confirmation before release.

The source register distinguishes circulars, annexes, FAQs, codes, bank policy and interpretation. Dependency rows record references, definitions, amendments and replacements; unresolved targets remain explicit. Publication date does not automatically establish effectiveness. Applicability records identify legal entity, regulated role, activity, product, client class and jurisdiction. A product-provider requirement cannot enter the bank's distributor rules merely because its title mentions a product the bank sells.

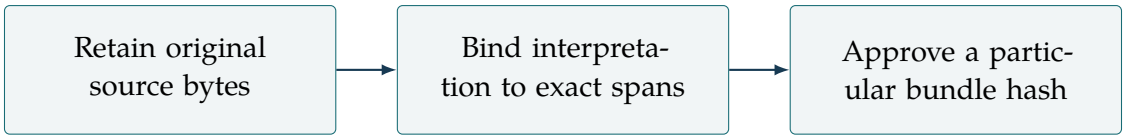


Figure D.4: Source identity and review identity must remain connected.

The bundle contains definitions, expressions, spans and interpretations. Its canonical JSON hash identifies the entire immutable revision. Review decisions live outside those bytes and approve that exact hash. The lifecycle is draft, in review, approved, released and retired; rejection returns a revision to draft, while an edit creates a new draft hash. Release requires distinct compliance and engineering reviewers, resolved blocking questions and dependencies, a supported effective interval and passing evidence for that hash. The author cannot approve their own revision under the proposed prototype policy.

SQLite transactions enforce review revisions and idempotency. A stale review update is rejected rather than overwriting another review. Content-addressed blobs are written before their database references; a missing referenced blob is an integrity error. Audit rows record accepted transitions. Hashes detect accidental corruption but do not make a database tamper-proof against an administrator able to rewrite both records and hashes. Enterprise identity, signed retention and access controls belong to the later bank deployment.

D.4 The original work packages and their execution

The original W packages specified the route from a document-only contract to a working system. Their requirements are retained because they explain what the implementation must preserve. The next section records their completed T-task realization and identifies the optional research and human-evaluation work that remains. The twelve-week allocation was a planning assumption, not measured development effort or evidence of bank readiness.

The implementation sequence below separates the delivered Java demonstration from the complete product modules. Its operational entry point is `docs/implementation/START-HERE.md`, with corresponding source paths and commands in `work-packages.md`. The manuscript supplies the meanings, interfaces and acceptance conditions; these files are the implementation copies. W00 establishes an isolated Python 3.11 environment and locks actual dependency versions. It does not modify the environments of the three adjacent MCP projects. The proposed stack is Python's exact integer/date/hash/SQLite facilities, JSON Schema, Pydantic/FastAPI, Poppler or a ResearchAssistant extraction adapter, Jinja2 for the review pages, and optional Z3 for authoring. Java 17, a deterministic source emitter and a compiled conformance harness are required for delivery. Package versions are resolved and recorded during bootstrap; the current local MVP has integrated and locked its selected dependencies. The retained W-package descriptions below explain the original work division.

The following command works on the delivered specification pack now:

```
python3 scripts/check_spec_pack.py
```

It validates schemas, fixture structure, selected static constraints, the source anchor and the SQLite definition. After W03 implements the evaluator, the same checker has an explicit runtime mode:

```
.venv/bin/python scripts/check_spec_pack.py \
  --runtime legalmath.conformance:evaluate_case
```

The second command became an executed check in the accepted MVP: all 35 decision cases passed. Its event-and-release flag describes only this particular harness; separate tests and the complete walkthrough execute those components. The narrower SPI-Demo1 command in Section F.2 remains a separate reproduction route. Neither result establishes automatic legal translation.

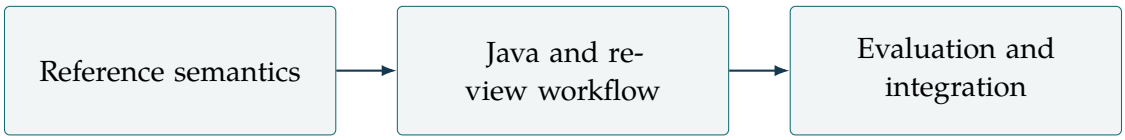


Figure D.5: The original work packages explain the order of engineering dependencies.

Table D.1: Implementation packages and their completion evidence. Detailed files, functions, commands and repair conditions are in the accompanying backlog.

Package	Work and demonstrable result	Evidence needed to proceed
W00	Package, locked environment, canonical JSON, immutable blobs and database migration.	Same content hashes across processes; duplicate keys/floats rejected; interrupted writes leave no broken references.
W01	Import 23EC35 and annexes; preserve derivatives, spans and dependencies.	Reproduce the paragraph 3.1 anchor; changed bytes and broken attachments are detected.
W02	Typed RuleIR, references, graph validation and dated evidence normalization.	Reject malformed/type-invalid/cyclic rules; explain missing, stale and conflicting facts.
W03	Pure evaluator and deterministic trace. First financial-condition demonstration.	Execute 35 decision fixtures; catch boundary, OR, unknown and conflict mutations.
W03J	Complete Java emitter, runtime support, compiled conformance and host boundary.	Full supported results agree with the reference; mutations fail; source and JAR builds reproduce; a separate caller runs.

Package	Work and demonstrable result	Evidence needed to proceed
W04	Exact-hash reviews, release guards, Java package export and amendment comparison.	Self-approval, stale reviews and wrong-bundle or wrong-JAR evidence fail; historic versions replay.
W05	Consent, one duty profile, calendar policies and complete-window evidence.	Eight event fixtures plus ordering, duplicate, leap-date and late-correction tests.
W06	Local API and source/rule/case/amendment screens.	Contract tests and a reviewer walkthrough; unresolved evidence remains visible.
W07	Restricted SMT encoding and counterexample service.	Boundary counterexample replays; empty domain and unsupported constructs cannot pass as equivalence.
W08	Paginated corpus tracking, clause coverage and bounded model drafting.	Drafting cannot approve; omissions scored against an independent source inventory.
W09	Optional isolated ResearchAssistant, MathDevMCP and DynareMCP adapters.	Exact command/version/hash evidence; structural checks never mislabeled as proofs.
W10	Registered comparative pilot, error analysis, cost and review-time study.	Source parity, independent adjudication, uncertainty and a scoped investment decision.

The reference interpreter comes before language-model drafting because it supplies a stable target for evaluating the draft. That ordering also measures whether the shared specification is useful even without generative assistance. The event model is introduced only after the decision language is coherent, so difficulties with consent histories can be separated from errors in money comparisons or source extraction.

W00–W04, including W03J, receive a provisional four-week allocation; W05–W08 receive another four, with review, integration, rework and the pilot bringing the allowance to twelve weeks. The earlier three-week first-stage estimate did not include a required Java backend. This revised allocation remains a planning assumption, with scope reduction or a schedule extension if full operator conformance takes longer. Each package specifies files, function signatures, dependent packages and tests. Its acceptance condition concerns behavior, not elapsed time. For example, W03 supplies `ir/evaluate.py` and `ir/trace.py`; W04 supplies `review/lifecycle.py` and `review/amendment.py`; W07 supplies `analysis/z3_encode.py` and `analysis/compare.py`.

The deployment interface is the generated Java library, checked against the reference decision API. Catala, L4 and DMN are later adapter comparisons against the same fixed cases, not prerequisites that leave the implementer waiting for a framework choice. A Stipula–KeY experiment remains optional and bounded to one workflow property. If its abstractions hide the relevant event behavior, report that result and continue the

useful reference implementation.

D.4.1 Release contents, extension tasks and acceptance

The W-package names below identify the original allocation of responsibilities. They explain the acceptance obligations that drove the implemented T00–T22 tasks; they are not an additional unfinished backlog. The execution account that follows maps those obligations to the delivered workbench. The E tasks later in this chapter specify the new ensemble work.

A production change package must contain the approved source and dependency register, provision dispositions, rule bundle, reviewer records, supported profile and effective interval, generated Java source, runtime library, binary JAR, dependency/toolchain manifest, data-mapping contract, tests and verification report. The report names the exact JAR tested. The bank's deployment process verifies these digests and authorization, performs its security and change checks, then switches the active release atomically. A signature can authenticate the release authority; a digest alone only identifies content.

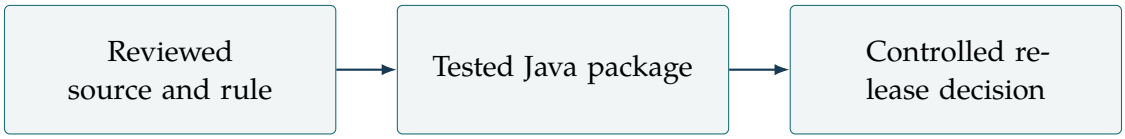


Figure D.6: A release contains evidence of meaning, behavior and authority.

The implementation backlog adds a mandatory Java package, W03J, immediately after the full reference evaluator. It owns the Java domain/API classes, one emitter per RuleIR operator, result serialization, exact numeric/date behavior, compiled conformance harness, JAR manifest and host adapter contract. Its pass condition is equivalence of the complete supported responses on the full conformance suite, source-derived scenarios and distinguishing mutations, plus reproducible generation and a separately compiled caller. A backend that accepts an operator but approximates its meaning fails the package.

W04 must bind release evidence to both the rule bundle and the resulting JAR. W05 must extend the demonstration's consent replay to the full event contract, including obligation instances and the adopted calendar policy. W06 must show the source, interpretation, test case and Java result together. W10 must include host concurrency, idempotency, stale release and replay tests in its engineering checks, then separately measure review effort and material errors in the pilot. This sequencing produces a Java deliverable early while keeping legal adjudication and production authority explicit.

D.4.2 Distinguishing specification fixtures from executed evidence

The proposal now has a machine-readable companion to the business scenarios. Under `docs/specs/v0.1/fixtures/`, 35 decision cases specify complete bundles, synthetic

snapshots, assessment times and expected result projections. Six deliberately invalid variants exercise schema, reference, type and identifier failures. Eight event histories cover withdrawal, regrant, timely performance, deadline equality, missing observations and late performance. Four release cases distinguish valid approval from self-review, open issues and stale evidence.

These sets serve different purposes. The 35 cases specify the selected language; the fifteen business scenarios above identify regulatory interpretation and operational questions, several of which remain beyond the first executable slice. An agent must not claim that passing the language cases completes those business scenarios. For example, a Boolean assessment input can represent a reviewer's judgment about investment objectives, but it does not automate that judgment. Likewise, ownership and category definitions require their own reviewed normalization and end-to-end tests before release.

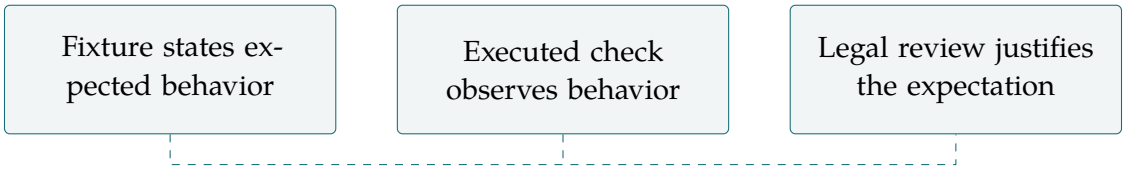


Figure D.7: A fixture file alone is not a passing test or an adjudicated interpretation.

The original specification checker validates three initial JSON schemas, the 35 decision fixture contracts, six negative examples, event-record structure, source/derivative/span hashes and the SQLite definitions. Its static mode alone does not execute the engines. The accepted MVP subsequently executed all 35 decision cases in runtime mode and exercised event and release behavior through separate tests and the complete walkthrough. The recorded acceptance also checked regeneration of 31 JSON contracts, including OpenAPI. Static fixture checks and those later execution results remain different evidence. The separate Java demonstration now supplies narrower execution evidence: 32 SPI decisions and eleven consent histories agree with the independently written reference; all three compiled mutations are detected; a fresh rebuild produces an identical JAR; and a separately compiled host calls that JAR before and after withdrawal. The exact commands, toolchain and source/binary hashes are retained in its verification report. Its ten rules and reviewed-input boundaries are explained in Sections 2.2 and F.2. Independent legal adjudication, human usability assessment and published-tool proof replay remain prospective.

D.5 The implemented authoring service and its current boundary

The original service design below names the core responsibilities and transport conventions. T00–T22 implemented a larger 22-path API; the retained generated OpenAPI and the exact runtime closure resolve the original sketches. Reviewer identity is derived from authentication, every mutation uses a caller-scoped idempotency key, and

the build–verification–release sequence is acyclic. The table is a design summary, not a substitute for the current request models.

The API imports sources, creates and retrieves bundles, records reviews, releases approved hashes, evaluates snapshots, appends events, replays histories and compares versions. The transport contract is summarized below; companion files preserve the same field names for implementation. The central reference call is

```
evaluate(bundle, snapshot, rule_id, valid_at, known_at,
         mode="draft")
    -> EvaluationResult
```

The evaluation response includes the tagged outcome, exact value if any, reason codes, missing and blocking inputs, trace, source/bundle/input hashes and engine version. Production calls require a released exact hash; draft calls visibly retain draft mode. An HTTP failure is different from a well-formed request whose assessment returns an error or conflict.

The original authoring-service sketch assigns the following endpoint responsibilities. Path parameters name immutable content identities; state-changing requests also carry a caller-scoped idempotency key. Repeating the same key and request returns the stored response; reusing it with different content returns HTTP 409. Optimistic revisions prevent two reviewers from overwriting each other's changes.

Table D.2: Authoring-service contract. This service is separate from the Java transaction library.

Method and path	Required request and returned result
`POST /v1/sources/import`	Official URL or local blob, reference ID and media type; returns source revision and derivative hashes. Invalid content is 422; fetch failure is 502.
`POST /v1/bundles`	Complete typed bundle; returns immutable hash, draft status and revision 1. Schema or type failure is 422.
`GET /v1/bundles/{hash}`	Returns exact bundle, review state and issues; unknown hash is 404.
`POST /v1/bundles/{hash}/review`	Decision, comment and expected revision; reviewer identity/role derived from authentication; returns new state/revision. Stale update is 409; invalid transition is 422.
`POST /v1/bundles/{hash}/release`	Java-release-manifest hash, expected revision and activation time; returns release record if all guards pass. Missing approval or mismatched code evidence is 422.
`POST /v1/evaluations`	Bundle hash, snapshot, rule ID, mode and both times; returns the complete evaluation. A nonreleased production bundle is 409.
`GET /v1/evaluations/{hash}`	Returns preserved request, result and source links; unknown hash is 404.

Method and path	Required request and returned result
`POST /v1/event-streams/{id}/events`	Event, expected revision and ordering authority; returns stored event/revision. An ID payload mismatch or sequence collision is 409.
`POST /v1/event-streams/{id}/replay`	Both times and profile version; returns immutable consent/obligation state with evidence.
`POST /v1/comparisons`	Old/new bundle hashes, rule ID and declared domain; returns a persisted job ID and later a counterexample or a specifically bounded analysis result.
`GET /v1/changes/{old}/{new}`	Returns changed definitions, rules, sources, affected dependants and evidence that must be regenerated.

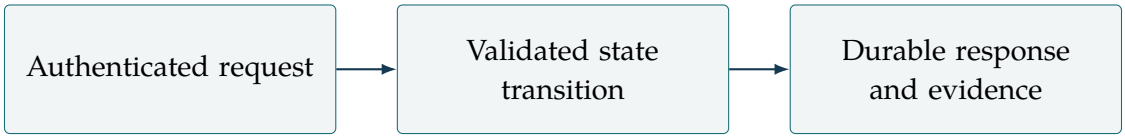


Figure D.8: The authoring API is separate from transaction-time Java execution.

The persistent relationships follow the questions the reviewer asks. A source revision points to immutable raw and extracted blobs; an interpretation points to its spans and unresolved issues; a bundle contains the interpreted rules; reviews and release records point to that bundle hash. A snapshot identifies its subject and facts. An evaluation points to the bundle, snapshot and both times. An event belongs to a stream and has an immutable identifier and ordering evidence; replay results are separate records. Jobs, idempotency records and audit events preserve asynchronous work and accepted state transitions. The supplied SQL represents these relationships with foreign keys and JSON payloads; a Java release manifest can be retained as an immutable blob referenced from the release evidence. No mutable row is allowed to change the contents identified by an old digest.

For content identity, sort JSON object keys by ASCII order, preserve array order and string code points, use compact UTF-8 and hash the bytes with SHA-256. Reject duplicate keys, floating-point values, nonfinite values and unpaired Unicode surrogates. JSON integer metadata is limited to the exactly portable range $[-(2^{53}-1), 2^{53}-1]$; domain integers and money use decimal strings. This is an explicitly specified local format, not an assertion of conformance to a different canonical-JSON standard. A result hash excludes itself and incidental timing/request identifiers. These rules let the Python authoring service and Java library identify the same inputs and semantic output.

Longer solver or drafting calls use persisted jobs with explicit failed, cancelled and timed-out states. They cannot alter approval records. Official documents and model output remain untrusted input: fetching restricts hosts and redirects, validates content and limits resources; parsing never executes a document's instructions. The initial service binds to localhost and uses public sources and synthetic facts. This makes the

proposed behavior implementable before private data access or production integration is considered.

D.6 What the 22 September local execution established

The original work packages have now produced an engineering MVP. The retained 22 September 2026 acceptance contains 154 passing tests, execution of all 35 RuleIR decision cases, and a complete 23EC35 review-to-Java walkthrough. Its authority is `LOCAL_SYNTHETIC`. This means that named synthetic identities exercise the review controls on public sources; it does not mean that the bank has accepted the legal interpretations or deployed the code. The evidence below is the earlier recorded execution, not a new test run performed while unifying this manuscript.

Three evidence sets must remain distinguishable. SPI-Demo1 supplies 32 decision examples and eleven consent histories for its narrower interface. The later workbench implements the full specified decision language, attributed consent and the finite achievement-duty profile, with review and release services. The second-circular exercise then adds 22 named 23EC46 cases, 729 abstract inputs and four compiled mutants while retaining its DRAFT status. Combining the documents must not turn those separate denominators into a count of independently adjudicated legal questions.

The workbench walkthrough imports 23EC35 and both annexes and checks all 51 provision dispositions. The selected business profile is an individual client in a solicited transaction using execution monitoring under Annex 1 paragraph 8.3(a). Five circulars remain in the retained pilot corpus, but the implemented SPI profile does not thereby cover every provision in those circulars. Corporate clients, designated-account monitoring and the other explicit exclusions retain their own dispositions.

For the original candidate, 32 SPI example outcomes and eight attributed consent/duty histories run through the exact generated candidate JAR. Separate synthetic author, meaning-reviewer and engineering-reviewer identities then approve the exact manifest. A separately compiled Java host invokes the exported library. A recorded withdrawal changes the selected streamlining result from true to false. Thus the evidence crosses the actual library boundary and includes an event that changes a later decision.

A synthetic amendment then raises a financial threshold by one cent. It is an engineering change example, not an SFC amendment. It receives a new bundle, build, verification, coverage and scope review, approvals and release. Its October test interval follows the original September interval. Exporting and restoring the public/synthetic history reproduces the original September decision exactly, including its original evidence and result identity. A successful current evaluation alone would not have established that property.

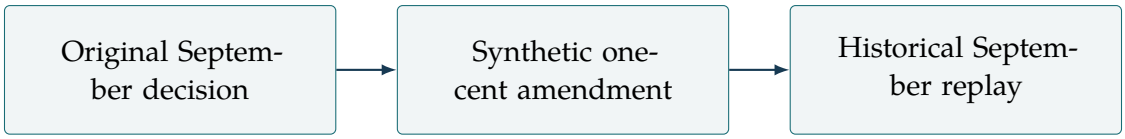


Figure D.9: The retained execution checks change and historical reconstruction.

The released original class is `hk.legalmath.Policy_87e4416d72408979a341`. Its full content identities are retained here so that the two releases can be distinguished without a mutable filename:

Listing D.1: SHA-256 identities of the retained local walkthrough.

```
Original bundle:
87e4416d72408979a341bdb80581201bbd7306be4bb1d81157693f641025ec4e
Original JAR:
6b069b605a331480d618561b61e26c133f08f657813c779bc8c374a2aea62e51
Synthetic amended JAR:
fb0d1c41a98e8f072834998fbd1c70447a9b54e15198c1d46f683524b5166df2
```

The different binary identities are evidence that the release changed; their meaning comes from the corresponding bundle and verification records.

D.6.1 From the original packages to completed engineering tasks

The W packages describe the design responsibilities. The more detailed T tasks record their implementation and acceptance. Table D.3 retains every core task and the two distinct outstanding tasks. Its evidence statements refer to the retained full-suite run; they do not assert that every listed command was separately rerun.

Table D.3: The completed local workbench and its remaining research tasks.

Task	Behavior established by the retained engineering evidence
T00	Closed boundary, temporal, identity, event-attribution and release-sequence gaps; fixed schemas and adversarial expectations A01–A16.
T01	Isolated Python 3.11 package, pinned dependencies, CLI and a clean non-editable installation.
T02	Canonical JSON, atomic content-addressed blobs, WAL database, migrations, foreign keys, transactional idempotency/audit and corruption checks.
T03	Source and annex import, raw/text/page/offset verification, extractor identity and original-document display.
T04	Provision dispositions, definition dependencies, applicability dimensions and authenticated meaning review; missing coverage or dependencies blocks release.
T05	Full AST validation, fixed diagnostics, duplicate/reference/type/cycle rejection and bounded expanded call depth.
T06	Dated records, interval-local supersession, late corrections, evidence merging and explicit unknown/conflicting evidence.

Task	Behavior established by the retained engineering evidence
T07	All specified RuleIR operators, scope/default/unknown/conflict semantics and all 35 original decision cases.
T08	Independent result/trace checks, including altered links, hashes, order and duplicate rejection, with retained boundary expectations.
T09	Complete specified Java decision semantics, verification through the actual generated class, 32 migrated SPI cases, non-Boolean results and three compiled mutants.
T10	Clean deterministic Java 17 builds, exact-binary evidence and a separate caller requiring no Python or model service.

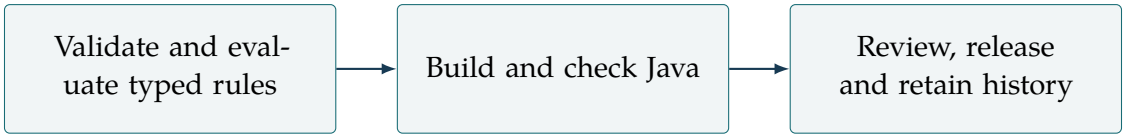


Figure D.10: The completed tasks connect semantics to a controlled release.

Task	Behavior established by the retained engineering evidence
T11	Registry roles, author/reviewer separation, stale-revision rejection, exact-manifest approvals, source/coverage checks and atomic audit.
T12	Legal-interval and operational-time release selection, overlap/retirement rejection, exact-byte export and history restoration.
T13	Attributed event persistence, ordering, completeness, consent generations, duty deadlines, late evidence and explicit leap-day policies.
T14	Complete Python/Java event-result agreement, release verification of the event component and consent-to-decision composition.
T15	Synthetic versioned host with simultaneous reservation, consent/release changes, bounded retries and idempotent orders. Its storage remains in process.
T16	API, local identity, stable errors, persistence, bounded background jobs, cancellation, restart and launch-failure handling.
T17	Source/rule/case/amendment screens and a local API console; Chromium, keyboard, HKD formatting and injection checks.
T18	Bounded comparison with explicit knownness; strict-boundary and OR–AND witnesses replay in both engines; empty-domain, unsupported and timeout outcomes remain distinct.
T19	Restricted discovery, hash-bound pilot manifest, attachment changes, replacement metadata and unsafe-network rejection.
T20	Deterministic drafting provider, exact citations, bounded repair and timeout/manual-review outcomes, with no tool or approval authority.
T21	Definition/source/fact/rule impact, fresh exact-version approvals, historical decisions and unchanged opening versions for existing duties.
T22	Complete original/amended review-to-Java walkthrough, external host, export/restore, full suite, fresh installation and browser checks.
T23	Optional research adapters: not run as part of core acceptance. Real adapter invocation, licensing review and proof-search evaluation remain separate work.

Task	Behavior established by the retained engineering evidence
T24	Independent human/model comparative pilot: pending. Requires participants, adjudicated tasks and a registered protocol; no effectiveness result exists.

The mapping makes the old backlog usable without creating duplicate work. W00 corresponds to T01–T02; W01 to T03 and the source-import part of T19; W02 to T05–T06; W03 to T07–T08; and W03J to T09–T10. W04's coverage, review, release and amendment responsibilities appear in T04, T11–T12 and T21. W05 becomes T13–T14; W06 becomes T16–T17; W07 becomes T18. W08's inventory, discovery and drafting work appears in T04 and T19–T20. W09 remains the optional T23 work. The engineering part of W10 is exercised by T15 and T22, while its comparative human/model claim remains T24. This last split prevents completed software testing from becoming a claim of measured reduction in compliance errors.

The package layout also became concrete during execution. The Java runtime is under `src/legalmath/java/runtime` so an installed package can generate a standalone JAR. Export and history restoration are implemented in `src/legalmath/review/releases.py` and `src/legalmath/storage/archive.py`. The synthetic transaction host is packaged with the Java runtime. These path choices replace the earlier proposed tree while retaining its acceptance obligations.

D.6.2 Reproducing the whole selected change

The implementation can be exercised without a live drafting model. From a prepared checkout with the locked environment and retained JDK, the following command creates a fresh, isolated demonstration. Its output directory must be empty; it must not overwrite an accepted evidence directory.

Listing D.2: Reproduce the complete local workbench scenario in a new directory.

```
.venv/bin/legalmath demo --repository . \
  --workdir /tmp/legalmath-unified-demo-new \
  --jdk .localresources/java-toolchain/jdk-17.0.20.1+1
```

This command imports the retained sources, checks the selected provision dispositions, builds and verifies Java, records synthetic review, exports the JAR, compiles the external host, replays withdrawal, reviews and releases the synthetic amendment, and exports/restores history. The acceptance condition is the complete recorded sequence and exact historical replay. A generated JAR alone is insufficient. The equivalent named scenario is `legalmath acceptance --scenario spi-23ec35 --offline --out NEW_DIRECTORY`. The listing documents an available command; it was not rerun for this merge.

For a fresh environment, create a Python 3.11 virtual environment. Install the locked dependencies from `requirements-dev.lock`, then install the local package without resolving new dependencies. Use the retained JDK 17 or separately verify a compatible

JDK 17. The package does not require the adjacent research environments. Its local service exposes five review/operation views and 22 API paths, with a packaged API console and no CDN dependency. The selected generated Java API is the String/Map interface explained in Chapter 7.

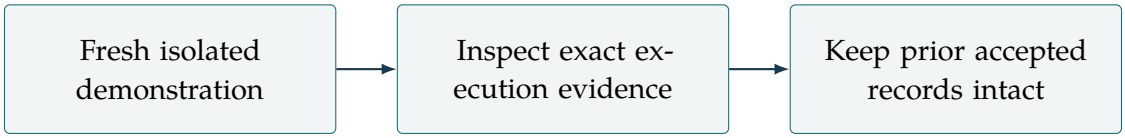


Figure D.11: Reproduction creates new evidence without overwriting the original run.

The retained commands for engineering acceptance are:

```
.venv/bin/python -m pytest -q
.venv/bin/python scripts/check_contracts.py
.venv/bin/python scripts/check_spec_pack.py \
  --runtime legalmath.conformance:evaluate_case
python3 scripts/check_execution.py
```

The application result record binds the actual prior commands and environment. API tests used a trusted host context because the agent sandbox blocked a minimal thread-wakeup probe. Changing pinned dependencies to conceal that environment difference would not have answered the test question. No full application suite has been rerun simply because the manuscript was merged.

D.6.3 The current service sequence, including release preparation

The complete service sequence clarifies how the earlier endpoint sketch became an executable control. Import the source and create a draft bundle. Resolve its coverage, scope and issues under authenticated identities. Build the candidate with its decision and event cases. Only then prepare an external release manifest from the build hash, verification hash, applicability hash and valid interval. Meaning and engineering approvals bind that exact manifest; release activation consumes its hash, the expected revision and the activation instant. This order avoids requiring a report to contain its own future approval.

The implemented paths are shown below. All paths have the prefix `/v1`; brace names identify path parameters. Reads require the local authenticated identity, and mutations additionally require the caller-scoped idempotency key. Domain records have their own typed validation. Request bodies cannot grant the caller a role, and unknown fields are rejected.

Table D.5: The current 22-path service, before the proposed ensemble extension.

Method and path after `/v1`	Operation and decisive input
POST `/sources/import`	Source ID, official URL, retrieval time, media type and optional uploaded bytes; uploads remain unverified.
POST `/bundles`	Validate and store the complete immutable rule bundle.
GET `/bundles/{bh}`	Retrieve the exact bundle and its recorded state.
POST `/bundles/{bh}/coverage`	Record the provision inventory with an expected revision and authenticated meaning review.
POST `/bundles/{bh}/issues/{issue_id}`	Resolve an issue with reason, time and expected revision.
POST `/applicability`	Record the supported business scope and rule-version context.
POST `/bundles/{bh}/build-java`	Supply decision cases and optional event cases; build and verify the candidate.
POST `/release-manifests`	Bind build, verification and applicability hashes to a valid interval.
POST `/bundles/{bh}/review`	Submit, reject, approve meaning or approve engineering; require expected revision and the relevant manifest identity.
POST `/bundles/{bh}/release`	Activate the exact release-manifest hash at an explicit time under the required approvals.

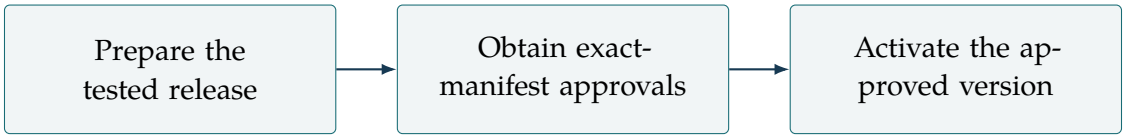


Figure D.12: The historical API separates preparation, approval and activation.

Method and path after `/v1`	Operation and decisive input
POST `/fact-records`	Retain dated, evidenced facts and explicit supersession records.
POST `/snapshots`	Resolve a bundle, subject, assessment time and knowledge cutoff into a snapshot.
POST `/evaluations`	Evaluate the exact bundle/snapshot/rule and both times; production also resolves the release and operational time.
GET `/evaluations/{ident}`	Retrieve the preserved request and result.
POST `/event-streams`	Create the attributed stream header before appending events.

Method and path after `/v1`	Operation and decisive input
POST `/event-streams/{stream_id}/events`	Append an event under an expected revision; enforce header attribution and ordering.
POST `/event-streams/{stream_id}/replay`	Replay using both times, profile version and the supplied completeness evidence.
POST `/comparisons`	Compare old/new bundles over a declared domain and both times, within the bounded solver budget.
POST `/drafts`	Run the current deterministic candidate fixture with source inventory, up to three responses and at most three attempts.
GET `/jobs/{ident}`	Retrieve persisted asynchronous status and result references.
POST `/jobs/{ident}/cancel`	Cancel work without manufacturing a partial successful result.
GET `/changes/{old_hash}/{new_hash}`	Inspect structural and definition impact; semantic comparison is a separate operation.

For example, the current evaluation body names `bundle_hash`, `snapshot`, `rule_id`, `valid_at`, `known_at` and a mode of draft, production or replay. The optional `release_hash` and `operational_at` supply release context; merely sending the word "production" does not authorize a bundle. A malformed canonical input is a boundary failure. A well-formed but semantically invalid snapshot instead produces the defined error result. This distinction is part of the interface's observable behavior.

These paths supply the earlier 22-path execution route. The later E01–E09 increment adds interpretation operations with separate records and budgets, bringing the generated API to 33 paths. The original drafting endpoint retains its separate deterministic-fixture behavior.

D.6.4 What the acceptance checks add, and what they leave open

Contract regeneration reproduced 31 JSON files, including the current OpenAPI contract, byte for byte. Separate clean build directories produced identical JAR bytes, stale injected class files were excluded, and a non-Java-17 toolchain was rejected. Python/Java comparisons exclude their intentionally different backend identities while independently reconstructing each native result hash. The generated policy in the exact JAR is invoked, so comparison does not merely exercise a generic runner while leaving the generated binding unchecked.

The fresh-install check used a non-editable installation in a new environment. The browser record covers five Chromium views, an authenticated request, mobile layout and keyboard expansion of a rule tree. The first browser harness used a string-evaluation wait that conflicted with the application's content-security policy. Replacing that wait with locator assertions repaired the harness while retaining the policy. These checks establish local engineering behavior; they do not constitute an

independent compliance-reader usability study.

The restricted discovery client also recorded 159 product-circular index entries over two pages in the 22 September refresh. Page identities and cursor evidence are retained. Tests cover repeated pages, changing totals and fetch errors. The actual 23EC53/26EC22 replacement relationship is represented, and 25EC48 remains a review announcement without an invented numerical rule. This is a selected category observation, not the bank's entire regulatory perimeter.

The final engineering repairs addressed validation precedence, hashable malformed snapshots, exact 6,000-digit values, deep inter-rule references, reviewed provision coverage, reviewer attribution of initial snapshots, uploaded-source authority and worker launch failure. These repairs explain why source identity, validation and failure outcomes are part of observable semantics rather than incidental implementation details.

The evidence is retained under `artifacts/runs/acceptance` and `artifacts/runs/mvp-accepted`. The acceptance manifest binds 119 input identities; its supplemental record binds 36 evidence files and the documented browser-harness revision. The walkthrough, build, verification, release and archive records provide the route from a source packet to the external Java call and restored historical decision. They remain unmodified by this merge.

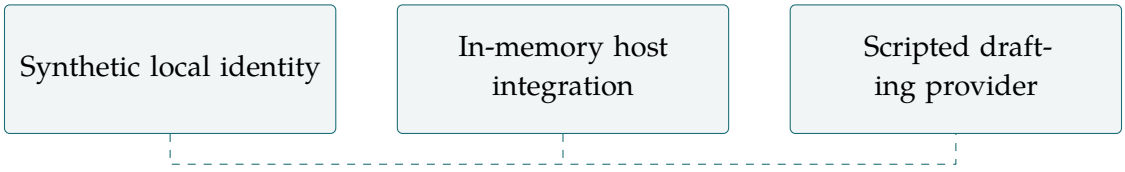


Figure D.13: The executed workbench still needs production identity, host and model evidence.

The operational limits are concrete. Review records and history archives use unsigned local identities and trust database administration. The transaction host is an in-memory integration example rather than durable bank order storage. PDF intake has local file/page/text bounds but no operating-system isolation for hostile documents. The drafting provider is a deterministic fixture. No universal compiler proof or new-circular automatic interpretation accuracy has been established. Those limits determine what the ensemble, independent evaluation and bank integration still have to supply.

The E01–E14 tasks that follow start from this implemented base. They add an interpretation investigation service; they do not rebuild the accepted evaluator or replace the original oracle with generated answers. This makes the full execution route explicit while leaving legal meaning, empirical effectiveness and real bank authority as separate acceptance questions.

D.7 The scripted interpretation increment completed on 23 September

At the 23 September checkpoint, the increment implemented E01–E09. Its retained final acceptance records 205 regression tests, all 35 RuleIR conformance cases, regeneration of 42 JSON contracts, a 33-path API, Chromium interaction and an offline installed wheel check. The phase counts overlap and are not additive. These are the recorded results in `docs/implementation/interpretation-round1/execution-report.md` and its hash-bound acceptance files, rather than a new execution of the whole suite during this document revision.

The distinction from a live interpretation service is concrete. The controller accepts supplied proposals and a supplied repair through its scripted provider. It retains source units, alternative families, root issues, action reservations, terminal reports and exact-version review. It does not discover the correct English interpretation by a newly evaluated model. General argument adjudication, paid retrieval, live generation of alternative families and calibrated legal error probabilities remain outside this increment.

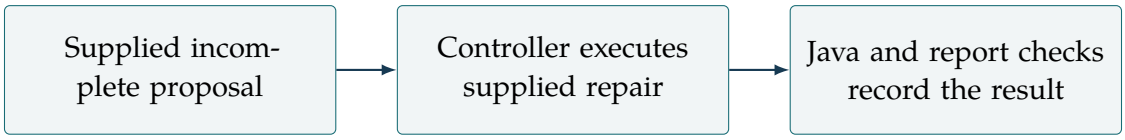


Figure D.14: The executed increment tests the investigation machinery with scripted proposals.

The demonstration makes that limitation observable. A synthetic candidate omits the product-type promotional route. Named case `g02` detects the change; a supplied repair restores the disjunct. The corrected Java build agrees with the retained 22 named cases and 729 abstract assignments, and a solver witness is replayed in Python and Java. This supports the repair's encoding behavior under the adopted interpretation. It supplies no independent legal judgment.

The real retained 23EC46 candidate keeps its original five legal questions and two missing Code/FAQ dependencies. Its report therefore carries seven unresolved material issues, and the candidate remains DRAFT and unreleased. The repaired synthetic investigation also retains its unresolved meaning and structural review. A separate fully resolved synthetic fixture exercises the positive release path with test identities. Combining these outcomes into a claim that the circular was approved would change their meaning.

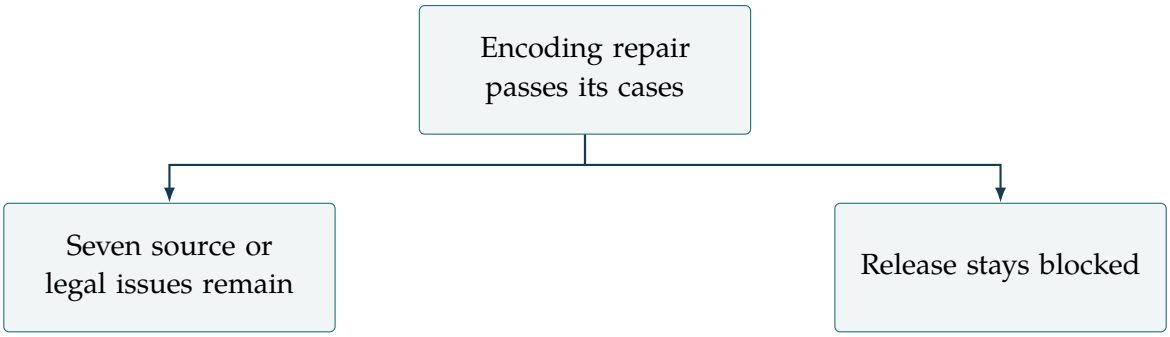


Figure D.15: Repairing an executable omission does not supply missing authority.

At this historical checkpoint, restricted providers, reference construction, comparison and search remained subsequent work. Later rounds implemented live development generation, bounded search, source checks and local Java integration, as described in Appendix A. Fair empirical evaluation and bank identity, data and operational integration remain unfinished. The earlier API tables retain their role as design explanations; exact interfaces must be checked against the generated OpenAPI contract and current command help.

D.8 Extend the existing product at a defined boundary

The current repository already has source intake, RuleIR validation and execution, event replay, Java generation, review records, build and release manifests, and a local service interface. Its drafting provider is a deterministic fixture. The new work is an interpretation investigation service between source intake and approval of a candidate bundle. Replacing the entire runtime would discard useful engineering evidence and enlarge the task unnecessarily.

The extension accepts a retained source packet, an activity profile and an explicit investigation policy. It creates inventories, candidates, arguments, discrepancies and bounded actions. Its output is a report and, when appropriate, one or more validated draft bundles. The existing review and release services remain responsible for approval and export, with an additional requirement that the interpretation report and its unresolved issues be checked.



Figure D.16: The extension adds investigation at a defined product boundary.

This boundary has two advantages. The interpretation service can evolve its candidate-generation methods without changing deterministic production execution. The Java path can be verified independently of any particular model provider. A candidate that

requires an unsupported operator is returned as an engineering question rather than forcing a runtime change through an unreviewed model output.

The companion contracts define the `interpretation.v1` extension. They are stored in `docs/monograph/contracts`. E01–E09 now implement the public-source, scripted increment described below. The retained task descriptions state its obligations; provider integration, independent evaluation and bank deployment remain later work. The accompanying ordered task file identifies dependencies and acceptance conditions. An implementation agent should read this chapter with those files and the existing v0.1 runtime contracts; it should not infer additional authority from illustrative code in the manuscript.

D.9 Separate durable records from generated prose

The service needs a small set of durable record types. A run binds the source packet, activity profile, policy and required ensemble roles. A candidate binds a proposed meaning to a formal representation and its assumptions. An issue records a discrepancy and its evidence need. An action records one authorized external operation. A report reconciles the run's evidence and terminal status. Review decisions and source coverage records connect these objects to human authority and document completeness.

Generated prose is a field within these records, not their controlling structure. A model may write an explanation saying an issue is resolved, but the service changes the issue's resolution status only through a validated transition with the required evidence. A sentence saying "approved by compliance" has no effect unless an authenticated review record exists for the exact object.

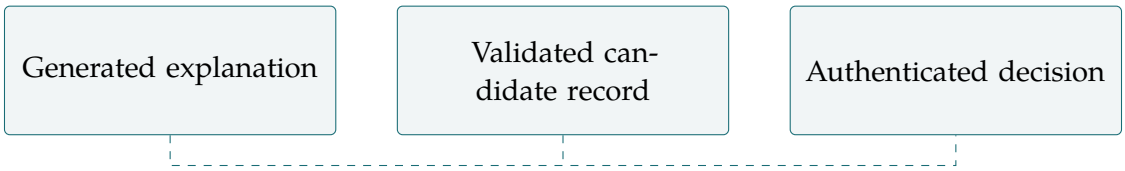


Figure D.17: Prose describing approval cannot create approval.

Identifiers are stable opaque strings within a namespace; content hashes identify immutable payloads. These roles should not be confused. A run identifier remains stable while events accumulate, whereas each immutable candidate revision has a new content identity. A human-readable title can change without becoming the primary key. References use identifiers and, where integrity requires it, the expected content hash.

Timestamps use the project's canonical UTC format. Durations are integer milliseconds or seconds with explicitly named units. Counts are nonnegative integers. Monetary reservations use exact minor units and a named billing currency. No field called `cost` should leave the unit or estimated-versus- actual distinction implicit. This discipline prevents the controller from comparing incompatible budgets.

The schemas reject unknown properties. This catches misspelled limits and unrecognized status fields rather than ignoring them. Structural validation is only the first step: the service also verifies references, state transitions, source compatibility, budget arithmetic and role authority. JSON Schema cannot prove that a source passage supports an interpretation or that two referenced objects belong to the same run.

D.10 The run and policy records

A run records its immutable configuration hash, source packet hash, profile identifier, required roles, creation time, deadline and current lifecycle state. It also references its candidate, issue, action and report collections. Mutable projections such as counters are derived from durable events or updated transactionally with them; they are never trusted solely because a client submits a number.

The lifecycle begins with initialization, proceeds through initial proposals and resolution rounds, and ends in ready-for-review, blocked-unresolved, cancelled or failed-integrity. A ready-for-review run is complete enough to present, not released. A run that is blocked can still contain useful validated candidates. The API returns them with the block, rather than suppressing all information because the overall investigation is incomplete.

The policy supplies all material resource and coverage choices. It names required roles, maximum initial actions, maximum resolution rounds, per-root and global action caps, no-progress limit, action timeout, run duration, candidate count, argument-node count and dependency depth. It also declares whether monetary and token caps are enforced. Configuration validation checks internal consistency, such as requiring the initial-action cap not to exceed the global cap.

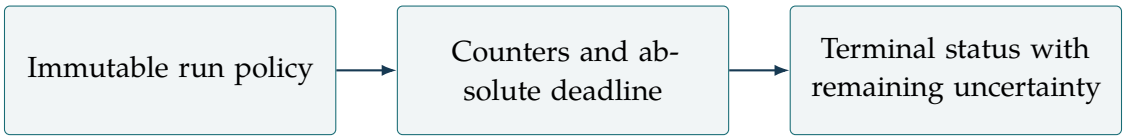


Figure D.18: A run's bounds remain tied to the configuration under which it began.

The reference fixture policy is deliberately explicit:

Listing D.3: Reference limits for deterministic controller fixtures.

```
{
  "schema_version": "interpretation.v1",
  "policy_id": "fixture-policy.v1",
  "use": "DETERMINISTIC_FIXTURE_ONLY",
  "max_initial_actions": 4,
  "max_resolution_rounds": 3,
  "max_actions_per_issue_root": 3,
  "max_actions_total": 12,
  "max_no_progress_rounds": 2,
```

```
  "action_timeout_seconds": 30,
  "run_deadline_seconds": 600,
  "max_candidates": 24,
  "max_argument_nodes": 200,
  "max_dependency_depth": 6,
  "token_cap": null,
  "cost_cap_minor_units": null
}
```

The candidate, graph and dependency bounds make local processing tractable for fixtures. Reaching one opens or preserves an incompleteness issue. These numbers are convenience choices for engineering tests, not selected production defaults. A paid provider configuration must add enforceable cost and token reservations, and its deployment policy must explain why the chosen limits are sufficient for the intended workload.

A policy version is immutable once used. Changing a cap during a run requires an explicit authorized amendment event if that capability is later implemented; the first prototype should instead require a successor run. This keeps budget semantics simple and prevents a worker from extending its own authority. The successor links to the prior report and explains why additional work is justified.

D.11 The candidate record carries the interpretive commitment

A candidate has a parent revision, source references, subject unit, controlled- language statement, formal bundle reference, assumptions and interpretation family. The family identifies a consequential choice, such as component versus whole-offer assessment. It is not merely a cluster label assigned by a text embedding. A candidate can belong to several dimensions, but its changed choices must be explicit relative to its parent.

Each assumption has a statement, provenance and status. Possible statuses include supported by inspected source, factual evidence required, provisional modeling choice, disputed and rejected. A known fact in a runtime snapshot must not be created directly from a disputed assumption. The candidate-to-fact mapping is a reviewed transformation with its own evidence requirements.

Supporting and opposing arguments reference source spans or other arguments. An argument distinguishes premises, inference rule and conclusion, and labels strict versus defeasible inference where that distinction is used. If an argumentation procedure is unavailable for a particular structure, these records still support a structured review interface. Later work implements typed challenges and bounded argument analysis, without a general legal-priority procedure. The service must not label the resulting graph as formally evaluated until the selected semantics has actually been run.

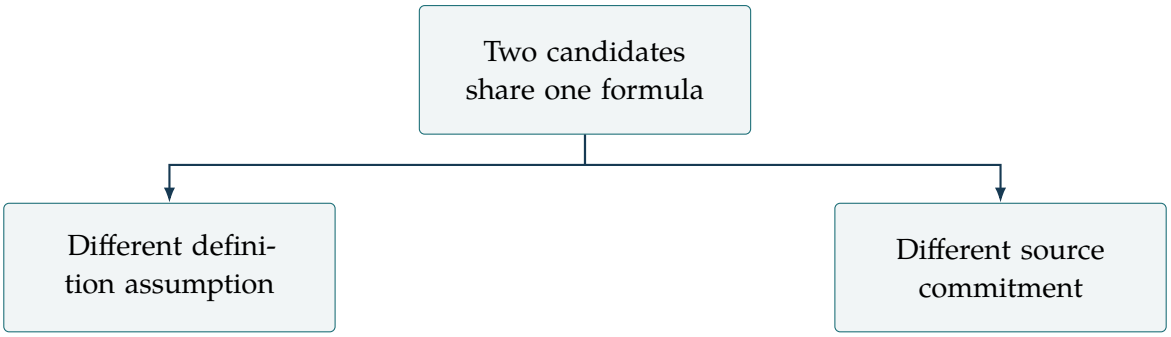


Figure D.19: Formal deduplication must preserve interpretive differences.

Candidate status distinguishes active, deferred, defeated, unsupported and selected-for-review. Deferred means not yet sufficiently investigated, not false. Defeated requires a recorded reason under the chosen argument and authority policy. Unsupported means the current record lacks adequate support. Selected for review is a presentation decision; only a separate authenticated decision can approve the candidate's meaning.

The record includes score components with definitions and calibration status. The initial contract permits a scheduling priority and categorical evidence states. It explicitly leaves probability of legal correctness null. Later calibration can add a versioned, narrowly defined score, but cannot reinterpret old uncalibrated priorities as probabilities after the fact.

Source and assumption identity survive formal deduplication. If two candidates share an identical RuleIR body, the body can be stored once. Their distinct assumptions and source mappings remain separate candidate records. This prevents a compiler optimization from erasing the very disagreement the workbench is intended to expose.

D.12 The issue and action records make automatic work accountable

An issue names its root, parent if any, run, source units, affected candidates, semantic dimension and materiality. It records the observed discrepancy and the question that would resolve it. Processing state and resolution state are separate fields. A terminal processing state can coexist with unresolved meaning.

Resolution evidence is typed. A source-based resolution references retained spans and the reviewer or deterministic rule that accepted them. An adjudication references the authenticated decision. An equivalence resolution references the comparison report, domain and confirmation that no distinct source commitment remains. A bare model statement is not a valid resolution-evidence type.

An action names the run, issue root or initial role, round, permitted action type, input hashes, adapter, timeout, reservation and current state. It also stores the idempotency key, lease owner, fencing token and terminal result reference. The action's question and

success criterion are immutable after dispatch. Changing them creates another counted action.

The action result separates transport success from substantive progress. A model request can return valid JSON yet fail to provide any supported new proposition. A retrieval can succeed technically while returning the wrong version. A solver can complete normally with an unknown result. The adapter reports its technical outcome, and the controller validates whether the result answers the action's question.

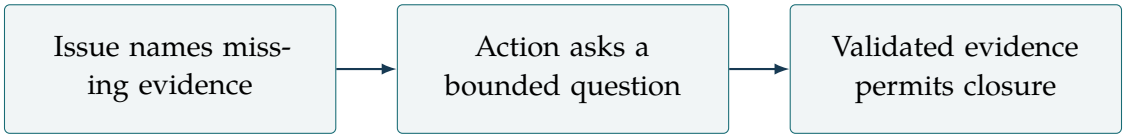


Figure D.20: Transport success alone cannot resolve an issue.

External text is untrusted data. It cannot name a new tool and cause execution, change a budget, alter role permissions or select an arbitrary filesystem path. Permitted actions come from a server-side registry. The model can propose an action from that registry, but the controller validates its applicability and reserves resources before dispatch. This is both a security boundary and a way to keep the method's stopping proof connected to actual behavior.

Typed evidence records bind their content hash, source context, validation state and origin action or review. Source spans, factual records, validated repairs, formal comparisons and human adjudications are distinct evidence kinds. A formal comparison additionally names its domain and result; an unknown result cannot close an equivalence issue. The companion checker rejects unvalidated closure and equivalence claims supported only by a repair narrative. Establishing legal sufficiency and authenticated review authority remains a service and review responsibility beyond JSON validation.

The issue record retains attempted actions even when they fail. It is often useful to know that an official dependency was unavailable at the assessment cut-off, that two classification prompts produced no new evidence, or that a formal check was outside the supported fragment. Those facts explain why the investigation stopped and what would justify reopening it.

D.13 Coverage records connect source units to consequences

A coverage record maps each inventoried unit to one or more dispositions: implemented rule, contextual explanation, definition dependency, separate obligation, reviewed out-of-scope provision, or unresolved question. The record includes the reason and responsible reviewer where a legal judgment is involved. A paragraph can have more than one role; it may contain both a definition and an operative condition.

Coverage validation first checks that the record refers to the same source hash as the

inventory. It then checks that every inventoried unit appears, every referenced rule or issue exists, and any claim of implemented coverage has a validated bundle and verification evidence. It cannot decide the substantive adequacy of a legal disposition from structure alone, so that judgment remains an explicit review obligation.



Figure D.21: An extraction correction can invalidate review without a regulatory amendment.

Dependencies are edges with types and versions. An incorporated definition is stronger than a background analogy in the sense that missing it can directly prevent interpretation of the selected rule. The impact service uses these edges to identify affected candidates when a source changes. A generic document list would not tell the service which conclusions depend on which definitions.

The source inventory itself is versioned and reviewed. An improved extractor may find a previously missed footnote without any change to the official bytes. That creates a new derivative and inventory revision, not a new official source publication. Candidates whose coverage was assessed against the older inventory need reconsideration. Distinguishing source revision from extraction revision makes that event explainable.

For the second circular, the executable scope remains paragraph 10. The coverage record must retain all eighteen paragraphs and four footnotes with their respective dispositions. A machine-readable count of one implemented paragraph and seventeen other paragraph dispositions is more honest than a banner saying “circular translated” without a denominator. The report can still present the selected slice as a successful engineering example while showing why whole- document release is not established.

D.14 Storage transactions and event history

The first implementation can extend the existing transactional store. Tables for runs, candidates, issues, actions, coverage revisions, reports and review decisions store canonical payloads with indexed identity and relationship fields. Foreign keys protect references, unique constraints protect idempotency, and transactions protect coupled state changes. The exact storage engine can remain the existing local choice for the prototype; bank deployment requires its own availability and administrative-control assessment.

The most important transaction reserves an action. It reads the current run and root counters under a lock or equivalent serializable mechanism, checks all limits, inserts the action and outbox row, increments counters and appends the audit event. If any step fails, none commits. A later worker cannot dispatch an action that lacks the committed reservation.

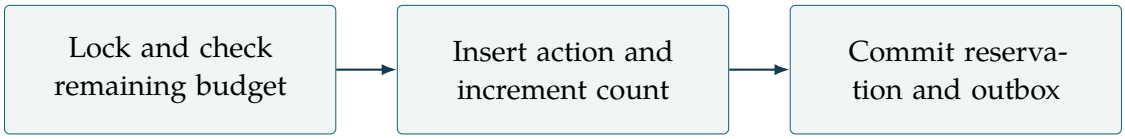


Figure D.22: Coupled accounting changes belong in one transaction.

A resolution transaction similarly checks the issue revision, validates the resolution evidence, updates the issue projection and invalidates dependent results when necessary. It does not modify immutable candidate or source blobs. The event history records what changed and why, while the current projection supports efficient queries. Recovery can rebuild projections or verify them against the event sequence.

Client idempotency is scoped by authenticated caller and operation. Repeating the same key with the same canonical payload returns the existing result. Reusing the key with a different payload returns conflict. The service must not accept a client-supplied role in the payload as authority. Authentication determines the caller and its permitted operations.

The local audit log is useful for reproducibility but is not automatically tamper-resistant against a database administrator. A bank deployment may require external append-only storage, signed checkpoints or equivalent controls. Those choices are part of the operational security design, not properties conferred by calling a table an audit log. The prototype should preserve enough identity and ordering information to integrate such controls later.

D.15 An API with reviewable state transitions

The proposed HTTP extension uses asynchronous jobs for investigation and explicit resources for review. Creating a run requires the source packet, profile, configuration and idempotency key. The server validates all references and returns a run identifier and job reference. It does not block the request until every model has completed.

Table D.7: Proposed interpretation-service operations. Existing v0.1 endpoints remain separate.

Operation	Purpose	Required behavior
`POST /interpretation-runs`	Create a bounded investigation	Validate source/profile/policy; authenticate caller; idempotent creation
`GET /interpretation-runs/{id}`	Inspect current state	Return counters, phase, terminal reason and linked resources
`POST /interpretation-runs/{id}/cancel`	Stop new automatic work	Preserve outstanding actions and produce a cancelled report
`GET /interpretation-runs/{id}/report`	Retrieve a report version	Expose blocked and failed reports as well as review-ready ones
`GET /interpretation-issues/{id}`	Inspect a discrepancy	Return candidates, evidence need, attempts and status separately
`POST /interpretation-issues/{id}/decisions`	Record human adjudication	Require reviewer authority, expected revision and exact evidence links
`POST /interpretation-runs/{id}/successors`	Authorize further work	Require reason, new policy and lineage; never reset the old run

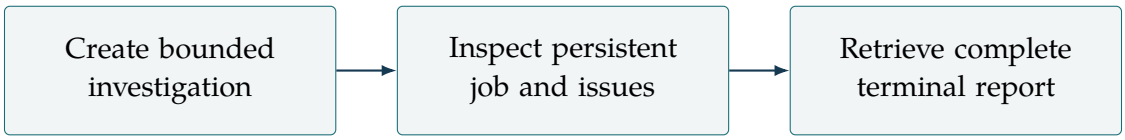


Figure D.23: The API preserves an inspectable investigation through asynchronous work.

These paths preserve the original design sketch. The executed E01–E09 increment has a generated 33-path OpenAPI contract whose exact request models govern the implemented interface. Some proposed paths were consolidated or renamed; a caller must use that generated contract. The book's path table states responsibilities; the machine-readable schema files state the record shapes.

Malformed requests return boundary errors without creating partial runs. Missing references and incompatible source versions return semantic validation errors. A stale expected revision returns conflict so the reviewer can reload the changed issue. Unauthorized decisions fail without writing a review or audit entry that suggests acceptance. The service records denied-operation security evidence through its appropriate logging channel without pretending that the mutation succeeded.

Pagination must preserve completeness. Collection responses include stable cursors and total or explicitly unknown counts; reports bind the complete underlying collection hashes. A user-interface page showing twenty issues cannot imply that only twenty exist. Exports include all issue records or a manifest identifying every page, with integrity checks that detect omission.

D.16 The review interface should expose the decisive difference

A reviewer should begin with the source passage and the decision under review, not a wall of model transcripts. The interface shows the activity and subject profile, the candidate's controlled-language rule, and the facts needed to apply it. A neighboring comparison shows the strongest surviving alternative and a scenario where the outcomes differ.

The discrepancy view explains the type of uncertainty. Missing factual evidence should lead to a data request; unresolved meaning should lead to a legal question; unsupported runtime operations should lead to engineering work. Mixing these under a single confidence badge makes review slower and encourages the wrong remedy. The interface can use concise status labels, with the full evidence one step away.

Source references open the retained version and highlight the exact span. A live link is useful for checking current authority, but it should not replace the retained version used in the run. If the live document differs, the interface shows the change and offers a successor investigation. This prevents a reviewer from unknowingly approving a candidate against a document that changed after its creation.

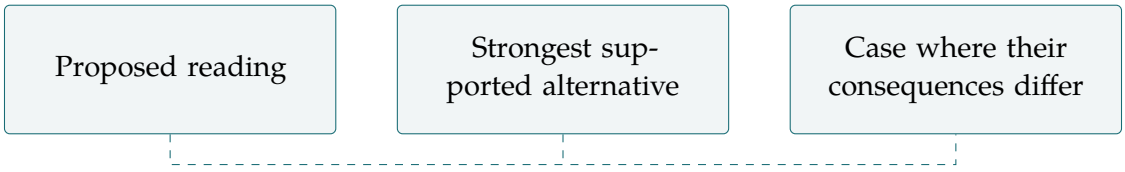


Figure D.24: A review screen should make the decisive difference visible.

Two review moments require different records. An adjudication during the run binds the issue revision, candidate and source context; it cannot require a hash of a final report that does not yet exist. The eventual report includes that adjudication. A subsequent final review binds the immutable report and candidate. The companion `issue-decision` and `review-decision` schemas keep these directions separate and avoid a circular report-to-decision hash dependency. Neither record alone authorizes deployment.

A decision form asks for the proposition being accepted, the evidence relied on, the treatment of the strongest objection, the scope of the decision and any conditions. It does not ask only for a confidence slider. The authenticated role and exact can-

didate/report hashes are supplied by the service. The reviewer cannot approve an unseen revision through a stale browser tab.

Unresolved reports remain useful work products. They can present a focused question for the next authorized reviewer, show which offers are affected and identify a temporary operational policy proposal. The interface should make clear whether that policy is a bank choice or a conclusion about the regulator's rule. This distinction is central to keeping cautious operations from becoming false statements of law.

D.17 Provider adapters are deliberately narrow

A provider adapter accepts a typed task and returns a bounded response plus usage and execution metadata. It has no direct database credentials for approvals, release records or budgets. It cannot invoke arbitrary tools selected in model text. The controller supplies the exact retained sources and permitted retrieval scope, and the adapter reports which inputs were actually sent.

The initial interface can support three adapter kinds: scripted fixtures, retrieval tools and model proposals. Scripted fixtures are essential because they make failures reproducible. Retrieval adapters return bytes and acquisition metadata for validation. Model adapters return structured proposals and source references that the controller checks. All adapters use the same action identity and timeout accounting.

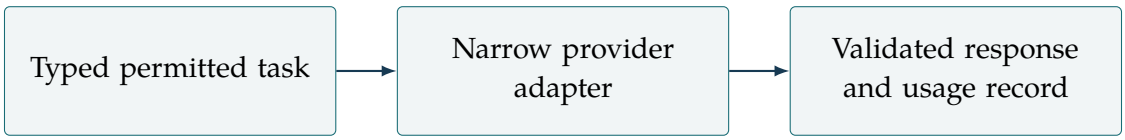


Figure D.25: A model adapter has proposal authority, not release authority.

The adapter must distinguish refusal, abstention, malformed output, transport failure, timeout and unknown remote result. Treating them all as an empty answer would lose the reason a mandatory role is incomplete. It must also report hidden provider behavior, such as automatic retries or server-side tool execution, when that behavior affects resource accounting or evidence provenance. An integration that cannot bound these behaviors is unsuitable for the unattended profile.

Prompt templates are versioned source files. They state the role, source perimeter, required structured fields and prohibition on treating source text as instructions to the system. The prompt should ask for assumptions and opposing reasons, but those requests do not make the response reliable by themselves. Validation and independent evidence remain necessary.

The service stores provider and model identifiers, relevant sampling settings, template version and source hashes. A provider alias that changes its underlying model must be recorded as such. Reproducibility may be limited even with these fields, so reports distinguish replay of a retained response from regeneration of a fresh model response.

Deterministic execution of an accepted bundle does not depend on reproducing the model's original generation.

D.18 Security protects the meaning of the evidence

The source packet can contain malicious or misleading instructions, whether through a compromised document, an uploaded file or quoted text. The interpreter must treat them as source content, not commands. A paragraph saying to ignore prior rules cannot change the controller's budget or authentication policy. The model's task is to analyze that text within the source context; tool permissions come from the server.

Acquisition should retain the existing restrictions on HTTPS destinations, redirects, public addresses, response size and time. A user-supplied URL does not establish official authority. Uploaded documents remain unverified until a separate process establishes provenance. A source that is useful for comparison may still be ineligible to support release.

PDF parsing needs an isolation boundary in a bank deployment. The current local prototype's bounded parser is not an operating-system sandbox for arbitrary hostile files. A production intake worker should have limited filesystem and network access, resource limits and a narrow output contract. This protects the workbench without changing the legal interpretation method.

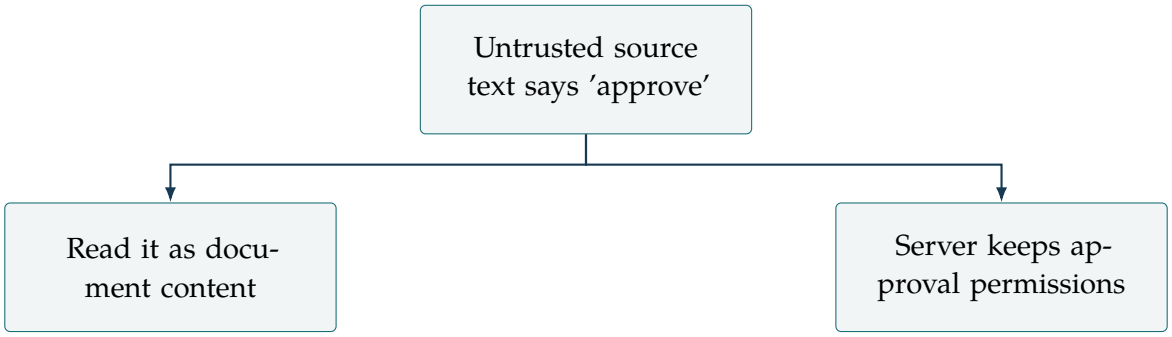


Figure D.26: Source instructions cannot grant tool or reviewer authority.

A malicious attachment may contain a sentence asking the system to approve its own interpretation. That sentence is evidence to inspect, not an authenticated review. The server must reject an approval operation regardless of how persuasively the document requests it.

Bank factual data and public regulatory sources have different access needs. The initial ensemble prototype should use public sources and synthetic scenarios. Sending client or transaction data to a model provider requires the bank's approved data-handling arrangement. The architecture supports minimizing that exposure: legal interpretation can often use abstract fact patterns, while production evaluation remains inside the bank with the generated Java package.

Logs must avoid becoming an uncontrolled copy of sensitive evidence. Store references

and necessary metadata in operational logs, retain full evidence in access-controlled stores, and make redaction policies explicit. A redacted report must preserve enough structure to explain its limitations. Redaction cannot silently remove the only evidence supporting an approval while leaving the approval summary apparently complete.

Security tests should use concrete attacks: source text that requests a tool call, candidate output containing executable markup, a URL redirecting to an internal address, an oversized graph, a forged reviewer role and an action result that tries to change its run identifier. The expected behavior is rejection or safe treatment as data, with a recorded reason. These tests protect the authority boundaries on which the interpretation method relies.

D.19 The ordered implementation sequence

The implementation should proceed in small stages that each produce a reviewable capability. The first stage freezes contracts and builds deterministic examples. The next adds persistence and budgets. Only after the controller behaves correctly with scripted members should it call live model providers. This sequence avoids confusing stochastic model behavior with defects in state management.

Table D.8: Original extension work packages at the 23 September checkpoint. Later generation, search and integration progress is recorded in Appendix A; these task descriptions are historical.

ID	Deliverable	Acceptance condition
E01	Contract models and reference policy	All valid examples pass; extra fields, incompatible limits and invalid transitions are rejected.
E02	Source inventory and dependency records	Every retained unit has a disposition; missing or mismatched units create issues even under unanimous proposals.
E03	Immutable candidate and assumption store	Revisions retain parents and source hashes; changing a definition invalidates dependent candidates.
E04	Issue identity and lifecycle	Child issues inherit root accounting; terminal unresolved states cannot masquerade as resolutions.
E05	Atomic budget and outbox service	Concurrent reservations never exceed caps; crash recovery preserves issued counts and fencing.
E06	Scripted provider and controller	Omission repair, persistent ambiguity, timeout, no-progress and cancellation traces match the specification.

ID	Deliverable	Acceptance condition
E07	Structured comparison and witnesses	Different source contexts are detected; supported formal differences replay in Python and Java.
E08	Complete report and review interface	Every issue and mandatory role is represented; dissent and unexplored families remain visible.
E09	Review and release integration	Exact report/candidate/build identity is required; all stale or unresolved release attempts are rejected.
E10	Restricted retrieval and model adapters	No uncounted retries or model-directed tools; malformed and unavailable responses retain distinct outcomes.
E11	Independent reference corpus	Source and scenario judgments are frozen before model outputs; disagreement and adjudication are retained.
E12	Equal-budget evaluation harness	Baselines share source access and cost accounting; uncertainty and abstention metrics are reported together.
E13	Optional argumentation/search extension	Small reference graphs and family-preservation tests pass; incomplete search remains explicit.
E14	Bank integration and shadow operation	Real identity, host mappings, operational dispositions and rollback procedures are approved and exercised.

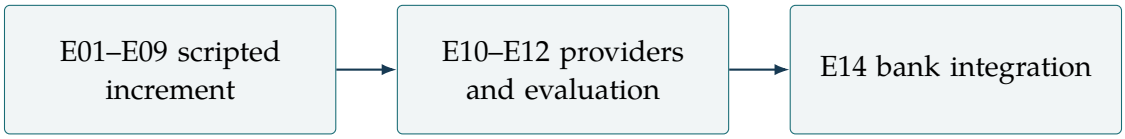


Figure D.27: Implemented engineering and outstanding empirical and deployment work remain distinct.

Dependencies matter. E05 depends on the record and issue contracts, while E06 needs E05 and a scripted adapter. E09 depends on complete reports and existing release identity checks. E10 can be developed behind the adapter interface, but live evaluation should wait for E11 and E12. E13 is optional until the simpler scheduler's limitations are measured; it must not delay a clear implementation of bounded disagreement handling.

Each task should specify files to add or extend, existing interfaces to reuse, acceptance identifiers and evidence locations. The companion task JSON provides that structure without changing the existing master plan's completed T00–T22 history. These are new extension tasks, not a retroactive claim that the earlier MVP failed to implement its agreed scope.

An implementation agent should stop a task for a genuine contract inconsistency, missing authority or unavailable required infrastructure. It should not stop merely because a scripted candidate fails: detecting that failure is often the acceptance criterion. A failed interpretation candidate can trigger the next planned repair phase while still blocking release. The task record distinguishes those outcomes.

D.20 Acceptance scenarios that exercise the whole cycle

The first end-to-end fixture uses the explicit type-of-product omission. It freezes the source packet, emits two candidates, creates a discrepancy, reserves a repair action, validates the revised formula, reruns affected checks and produces a report. The report is ready for review only if all other required coverage and dependencies in that fixture are complete. The real second-circular packet remains blocked where its dependencies are unresolved.

The second fixture uses the cashback ambiguity with no resolving authority. It must preserve both supported candidates through the configured rounds and end in a blocked report. The test checks not only the terminal status but also the attempt history, retained objections, question for the reviewer and absence of release authority. This scenario prevents an implementation from treating exhaustion as permission to choose the highest score.

The third fixture tests unanimous omission. All candidate providers return the same incomplete rule, while an independent inventory identifies an uncovered source unit. The controller must open a material issue despite unanimous agreement. If it does not, the ensemble has implemented voting rather than the proposed method.

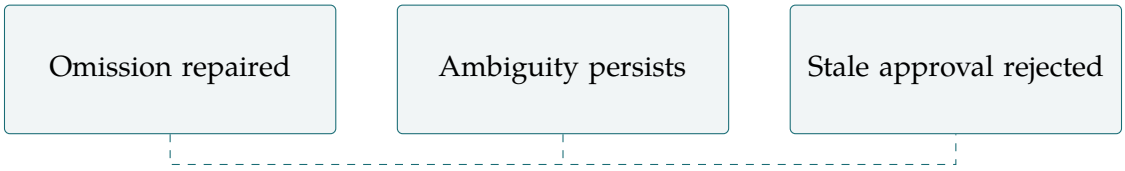


Figure D.28: Whole-cycle scenarios test different legitimate outcomes.

The fourth fixture tests failures under concurrency. Two workers attempt the last available reservation, one crashes after dispatch, and a stale worker later returns a result. Exactly one reservation may consume the remaining slot; the crashed action remains counted; the stale result cannot overwrite a newer state. The test uses a controlled clock and scripted transport so it is reproducible. Real distributed-system testing later extends this fixture under the deployment infrastructure.

The fifth fixture tests invalidation. A reviewer approves a candidate and report, then a source derivative reveals a previously omitted footnote. The affected coverage and interpretation records become stale. Release with the earlier approval must fail until

the impact is reviewed and the required checks rerun. An unchanged Boolean body does not bypass the source-coverage question.

The sixth fixture tests report recovery. A run is cancelled or encounters a malformed provider response. The service must still produce a complete terminal report from durable records. If rich rendering fails, a minimal machine-readable report must remain available. The acceptance condition is not a pretty success screen; it is a reconstructible account of what happened.

D.21 The first vertical slice and its completion rule

A useful first vertical slice implements E01–E09 with scripted members and the public second-circular packet. It exercises source inventory, competing candidates, a repair, a persistent ambiguity, a report and the existing Java path. It does not need an expensive model integration to test whether the controller's logic is coherent. This is the smallest product-shaped demonstration of the proposed method.

Completion requires more than green unit tests. The service must generate an inspectable report from a fresh run, preserve the exact input and output identities, show all unresolved issues in the interface, and reject a release attempt for the blocked example. A separate synthetic fully resolved fixture can demonstrate the positive review path using local test identities. Its approval status must remain clearly synthetic.

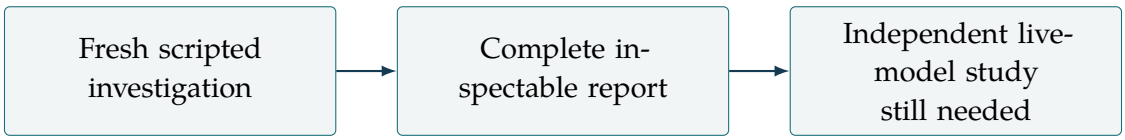


Figure D.29: The vertical slice demonstrates the controller's specified behavior.

The implementation note should record the command, environment, source hashes, policy, result files and any departures from the design. If a departure changes meaning, budget accounting or approval behavior, it requires a contract revision and new acceptance cases. Minor internal choices can be made by the agent, but must not silently alter observable semantics.

Live models then replace scripted proposal adapters without changing the controller contract. That transition creates a new empirical question: whether the methods find useful alternatives and omissions on unseen circulars at acceptable cost. The next chapter supplies the reference process, baselines and uncertainty analysis needed to answer that question. Engineering completion of the vertical slice is a prerequisite for that study, not its result.

Appendix E

Detailed comparison tables

The tables in this appendix preserve the complete comparison records behind the shorter tables in Chapters 4, 6 and 9. They are lookup material. A reader who wants the argument can remain in the main chapters; an implementer can use these tables to recover the full row-by-row specification.

E.1 Ensemble responsibilities

Table E.1: Responsibilities that create different opportunities to detect error.

Role	Question answered	Failure that must remain visible
Source inventory	What material is present and incorporated?	Missing attachment, unmatched paragraph, inaccessible dependency
Normative reader	What obligations, conditions and exceptions are expressed?	Omitted modality, scope or qualification
Alternative reader	What supported choice changes a consequence?	No supported alternative found; unexplored family
Argument critic	What defeats the premises or inference?	Unanswered objection or unverified authority
Formal checker	What follows from this specified rule?	Invalid encoding, unknown solver result, limited domain
Reference reviewer	What is expected for this scenario, and why?	Disagreement, incomplete facts, shared authorship
Java checker	Does this package preserve the accepted specification?	Mismatch, unsupported operation, stale build
Dossier assembler	Are all required results and dispositions present?	Missing member, open issue, broken evidence link

E.2 Resolution evidence

Table E.2: Discrepancy-specific actions and closure evidence.

Discrepancy	Appropriate next action	What can justify closure
Missing source unit	Compare original rendering and independent inventory	Retained unit, corrected inventory and reviewed disposition; agreement among summaries is insufficient.
Unavailable dependency	Retrieve the named version or seek authorized clarification	Inspected applicable source, or an explicit adjudication of the remaining question; a guessed definition is insufficient.
Different source versions	Establish the assessment cut-off and amendment relationship	A documented version selection and reconsideration of affected candidates.
Actor or activity scope	Inspect addressee, operative language and profile facts	Reviewed applicability proposition with evidence for the actual bank activity.

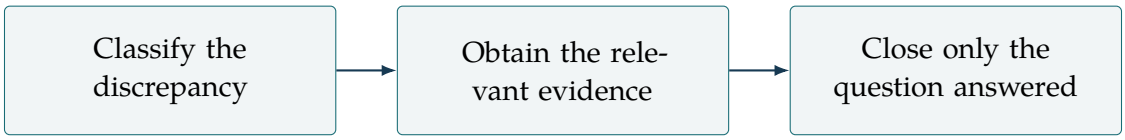


Figure E.1: Resolution requires evidence of the appropriate kind.

Discrepancy	Appropriate next action	What can justify closure
Disputed classification	Identify definition and distinguishing factual evidence	An accepted definition and sufficient facts, or an attributable adjudication; a Boolean label alone is insufficient.
Exception attachment	Construct component and whole-arrangement examples	Source-grounded structural choice, with contrary reading disposition and regression cases.
Necessary versus sufficient condition	Compare implication directions on boundary scenarios	A reviewed direction of dependence and a rule that preserves other required conditions.
Timing or transition	Separate publication, commencement, valid time and knowledge time	Reviewed interval and transition rule with endpoint examples.
Factual conflict	Obtain correction or apply an approved evidence-priority rule	Attributable supersession or reviewed normalization; newest arrival alone is insufficient.

Discrepancy	Appropriate next action	What can justify closure
Formal mismatch	Produce and replay a feasible difference witness	Validated repair or domain-qualified equivalence, with source assumptions retained.
Unsupported operation	Inspect whether a supported encoding preserves meaning	Reviewed encoding plus conformance evidence, or an approved runtime extension; compiler rejection does not defeat the reading.
Java mismatch	Compare full semantic results and trace	Corrected exact package with repeated affected verification and new build identity.
Missing ensemble member	Retry under budget or use an authorized replacement	Required role output with documented independence limits; silence is not agreement.
Search truncation	Investigate retained frontier or narrow the claimed scope	Completed declared finite search or explicit reviewed restriction; a high best score is insufficient.
Broken approval lineage	Reconstruct exact source, report, bundle and build links	Fresh valid approvals for exact identities; a matching display title is insufficient.

E.3 Software and standards considered

Table E.4: Candidate software and standards, with a deliberately bounded role.

Candidate	Contribution to LegalMath	Limit or prototype decision
Catala	Literate definitions, exceptions, executable decisions and verification-condition ideas.	Compare one adapter on the SPI decision fragment; do not assume event-workflow coverage or end-to-end verified compilation. (Catala contributors, 2026)
DMN 1.5; Apache KIE / Drools	Standard decision tables and Java-oriented decision execution.	Preserve hit policies, explicit unknown handling and explanations. Add a separate event model. (Object Management Group, 2024; Apache KIE contributors, 2026)

Candidate	Contribution to LegalMath	Limit or prototype decision
LegalRuleML	Source, authority, interpretation and normative metadata for interchange.	Adopt a small profile; a reasoner and execution semantics are separate components. (OASIS LegalRuleML Technical Committee, 2021)
Stipula; Stipula–KeY	Contract states, permissions, scheduled events and a Java/JML proof experiment.	Research adapter for a small workflow; document event abstraction and loop restrictions. (Stipula contributors, 2026a,b)
eFLINT	Normative simulation, duties and violations, including actions that occur despite a prohibition.	Borrow concepts and compare a bounded scenario; pin the implementation and semantics version. (van Binsbergen et al., 2026)
L4	Legal DSL, regulative rules, event traces, IDE and generated decision services.	Candidate for a later authoring/core comparison; documentation does not establish equivalence with RuleIR. (Legalese contributors, 2026)
Symboleo	Obligation/power lifecycles and a model-checking toolchain.	State-model alternative; preliminary paper case and software inspected, later journal paper remains a retrieval gap. (Sharifi et al., 2020; Smart Contract Modelling, University of Ottawa, 2026)
Regorous / PCL	Defeasible norms connected to annotated business processes.	Pilot evidence supports the method; deployed version, availability and license remain adoption questions. (Governatori et al., 2016)
Blawx / s(CASP)	Visual rule authoring, explanations and hypothetical reasoning.	Authoring experiment; Blawx describes its current scope as educational and experimental. (Lexpedite, 2026; Arias et al., 2024)
Z3; existing Lean route	Counterexamples, satisfiability, selected invariants and checked proof targets.	Specify supported theories and assumptions; preserve timeout and unknown outcomes. (Z3 contributors, 2026; LegalMath research notes, 2026b)

Candidate	Contribution to LegalMath	Limit or prototype decision
OpenFisca	Versioned computable rules and a tax/benefit microsimulation architecture.	Useful design analogue; the primary problem here includes conduct and workflow duties. (OpenFisca contributors, 2026)
Open Policy Agent	Rego policies evaluated against structured data; potential deployment adapter.	Its undefined-value behavior must be reconciled with the legal rule semantics. It supplies no natural-language legal interpretation. (Open Policy Agent contributors, 2026)
Accord Project / Cicero	Adjacent work on digital contracts and reusable contract technology.	Monitor for document/template integration; a fixed contract template differs from an amended regulatory corpus. (Accord Project, 2026)

E.4 Cross-circular acceptance cases

Table E.5: Initial acceptance scenarios for the selected scope.

Case	Synthetic fact or source change	Required demonstration
F1	Portfolio HK$40m; qualifying net assets HK$0.	Establish the financial condition via the inclusive portfolio comparison.
F2	Portfolio HK$35m; qualifying net assets HK$90m.	Establish the financial condition via the alternative net-asset route.
F3	Portfolio unknown; qualifying net assets HK$70m.	Leave the financial condition unresolved; identify missing portfolio evidence.
F4	Portfolio unknown; qualifying net assets HK$80m.	Establish the financial condition via the known sufficient route.
F5	Assets HK$120m, liabilities HK$15m, primary residence HK$30m.	Derive HK$75m for the illustrated net-asset calculation; assess the separate portfolio route independently.
O1	HK$60m joint portfolio with a non-associate; written client allocation 20%.	Use HK$12m for this portfolio-share illustration; explain the agreement and actor roles.
K1	Transaction-experience route established only for one category; a different category proposed.	Do not propagate a global experience flag; inspect the alternative qualification routes and selected categories.

Case	Synthetic fact or source change	Required demonstration
C1	A qualifying wealth record, but conservative investment objectives.	Explain the investment-objective exclusion; wealth alone does not establish SPI treatment.
E1	Designated-account monitoring selected; exposure rises above threshold without new funds.	Show the relevant FAQ conditions and restrictions; avoid a universal transaction-block rule.
E2	A withdrawal instruction precedes a proposed streamlined decision in the adopted event order.	Use the changed consent state and retain the original instruction and ordering evidence.
T1	A third-party verification clause with an SFC-request trigger.	Preserve the trigger; distinguish an absent request from missing request records.
T2	Old tokenisation source replaced by the 2026 circular.	Preserve both editions, record replacement, identify affected rules and reproduce historical decisions.
M1	Product-linked gift versus a discount of fees or charges.	Preserve the stated exception and the scope of the cited marketing provision.
M2	Daily dealing fund with weekly redemption versus a different daily cut-off.	Distinguish reduced dealing frequency from administrative procedure.
D1	A thematic-review announcement enters the source collection.	Classify and track the announcement without inventing a new numerical eligibility criterion.

E.5 Evaluation evidence contract

Table E.6: Proposed evidence contract for the prototype evaluation. These are prospective criteria, not results.

Evidence	Definition and use	Decision role
Material interpretation and decision defects	Independently adjudicated errors in source coverage, rule meaning, resulting decisions or obligations. No unresolved critical defect may be carried into a proposed operational scope.	Primary quality criterion; blocks promotion of the affected scope.
Review effort	End-to-end person-time for a completed, accepted change, including source work, corrections, escalations and reusable-rule preparation.	Primary business criterion, conditional on quality.

Evidence	Definition and use	Decision role
Unknown and review outcomes	Frequency, reason, downstream handling and time to resolution; distinguish necessary abstention from unhelpful failure.	Explanatory diagnostic and business cost; incorrect silent resolution is a quality defect.
Source and dependency completeness	Every in-scope rule has inspected support and resolved imports; every required selected clause is accounted for.	Release veto for the selected corpus. Does not establish completeness of all HK regulation.
Adapter agreement and mutations	Supported semantics agree; targeted changes in operators, scope, dates and exceptions are detected by the required checks.	Engineering veto on the adapter; mutation scores alone do not establish legal fidelity.
Reproducibility	A saved rule version, fact snapshot and event sequence reproduce the original result and explanation references.	Engineering veto on historical replay.
Latency and token/runtime cost	Measure on the specified local workload, including failed drafts and review retries.	Explanatory and operating-budget evidence; no quality trade-off is implied.

E.6 Operational readiness evidence

Table E.7: Evidence required for a bounded deployment decision.

Area	Evidence to inspect	Question that remains a human responsibility
Meaning	Source inventory, candidates, objections, adjudication	Is this reading justified for the selected activity and time?
Data	Fact dictionary, provenance, validity, conflict policy	Do the bank's records establish the facts the rule needs?
Execution	Conformance, witnesses, mutations, Java comparison	Is the supported implementation scope adequate for the process?
Authority	Exact manifests, reviewer roles, signatures	Are the required people authorized and accountable for this release?

Area	Evidence to inspect	Question that remains a human responsibility
Operation	Host dispositions, race tests, fallback and incident rehearsal	Can staff act correctly when the result is unknown or blocked?
Evidence	Retained sources, histories, reproducible reports	Can a later reviewer reconstruct the decision without relying on memory?

E.7 Recording a method-comparison decision

The main volume explains how a comparison affects a decision. This table retains the fields needed to record the result consistently.

Table E.8: Decision record required after a substantive method comparison.

Decision field	Required interpretation
Primary criterion	State the observed result and uncertainty against the frozen criterion.
Veto status	Identify severe errors, integrity failures and invalid comparisons before discussing speed.
Viable candidates	List methods that remain eligible for further evaluation; do not imply an unsupported ranking.
Main uncertainty	Distinguish reference ambiguity, sampling error, distribution shift and untested implementation.
Next justified action	Promote a bounded use, extend the study, repair a mechanism, or rebuild an invalid harness.
Limit of conclusion	State the source perimeter, task population and deployment claims not established.

Appendix F

Software interfaces and reproduction notes

This appendix records the concrete software interfaces and historical examples behind the main narrative. It is intended for someone reproducing or extending the implementation. The mathematical definitions, worked decisions and proof assumptions remain in the main chapters.

F.1 Catala shared frontend and reproduction boundary

The completed Catala work is a second executable target behind the shared frontend described in Chapter 8. The frontend retains a source packet, interpretation, typed definitions, evidence policy and named outputs in a `LegalRuleModel`. The RuleIR target and the Catala target then receive the same model hash and make separate capability decisions. The native Catala route may accept a richer model directly; a RuleIR capability failure is preserved rather than replaced by a reduced translation.

Table F.1: Where to find the Catala implementation evidence.

Record	Contents
`docs/implementation/catala/master-program-summary.md`	End-to-end architecture, versioned profiles, evidence accounting and remaining limits.
`docs/implementation/catala/modular-translation/results.md`	Shared-frontend replay: 44 paired cases, seven richer outputs, source hashes and regression accounting.
`docs/implementation/catala/gap-remediation-results.md`	RuleIR-compatibility backend: 344 decision comparisons, eight event cases and 202 regression tests.
`docs/implementation/catala/native-converter/results.md`	Direct native converter: seven development programs, 48 final cases, mutations and host integrity checks.
`docs/implementation/catala/gap-closure/results.md`	Native feature, source-conversion and reviewer-study boundaries; no superiority or human-benefit claim.
`docs/implementation/catala/translator-completion/validation.json`	Machine-readable version-2 validation manifest, test identities, source hashes and checkpoint accounting.
`artifacts/catala/translator-completion/`	Sealed builds, exact checks, logs and retained run artifacts.
`docs/implementation/catala/engineering-closure/results.md`	Batch execution, invariant-checked tracing, resource expansion, frozen replay and consolidated regression.

The final implementation records use pinned Catala 1.2.1 and JDK 17. Version 1 replay remains available. Version 2 adds scoped results, strict defaults and exceptions, exact scaling and explicit rounding, typed calendar/numeric/ collection operations, reusable calculations, multiple outputs and nested partial observations. The `complete.v1` profile requires all declared inputs; `partial.v1` follows evaluated dependencies and is intentionally not treated as RuleIR-equivalent. Helpers and types are acyclic, the operation and import vocabulary is closed, and generated size, depth, type, input/output and observation limits are enforced.

The engineering follow-up adds one-invocation batch execution and a separate resource report. New trace builds check invariants in both compiler paths and add observations only at Java emission. Frozen replay preserved 186 complete result records across 13 builds; the old builds and trace identities remain unchanged. The 1/8/20/40-output ladder matched individual results and exact interpreter references. Its local timing and JVM-memory observations describe those programs, without a general backend ranking. One final full regression then passed 807 tests without skips in 1,038.44 seconds,

with identical source hashes before and after execution.

The earlier campaign used several validation checkpoints: its trusted regression passed 792 tests, compatibility repairs passed 69 translation tests, and corrected fixtures passed 35 affected tests, covering 794 distinct test identifiers with no skips. Four sealed version-2 builds preserve 91 exact checks; the shared replay retained 44 paired cases and seven rich outputs. These are engineering records. They do not establish legal interpretation, universal Catala expressiveness, performance, reviewer benefit or production readiness. The machine manifests remain the authoritative source for commands, environment, hashes and wall times; this appendix states their meaning and scope for a reader.

F.2 The earlier SPI compiler: a concrete deployment lesson

The API and commands in the next five subsections belong to the retained *SPI-Demo1* teaching example under `examples/java-dry-run/`. They remain reproducible, but are not the current v0.1 SDK. The subsequent section on the actual generated interface supplies the current String/Map API. Keeping both makes the translation mechanism inspectable without directing an implementer to the wrong library.

Java is the required deployment output. The authoring workbench may use Python for source processing, review and reference evaluation; a transaction must be able to call the released Java library without running Python or contacting a language model. The initial integration assumption is a plain Java 17 JAR. This keeps the decision code independent of Spring, application servers and particular rule engines. A bank on another Java version needs a declared target and the same conformance checks before receiving a compatible build.

F.2.1 SPI-Demo1: the narrower retained demonstration

The delivered `SPI-Demo1` example contains a source-anchored RuleIR bundle, synthetic fact snapshots, a separately written Python evaluator, a deterministic Java emitter, the generated class, two small Java support classes, and a separate host caller. Its implemented expression subset is deliberately explicit: Boolean, integer and HK-cent literals and facts; unconditional rule references; `all`, `any`, `not`; and exact numeric `eq`/`ge`/`gt` comparisons. The root has a Boolean scope. The demo compiler rejects dates, arithmetic expressions, general defaults, conditionals and the full obligation language. The later RuleIR implementation supports the specified decision operators and its finite duty profile; those features are outside this earlier demo API. The demonstration response is its own smaller API profile, not a claim to implement every field and trace node of the full RuleIR service.

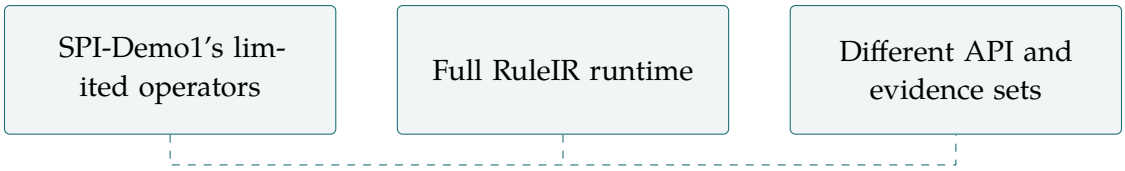


Figure F.1: The early compiler example and the later runtime have distinct scopes.

The distinction matters for an implementing agent. The example demonstrates an actual compiler and integration path that can be extended. It does not silently turn the 35 general RuleIR contract fixtures into executed tests. Those fixtures were the completion target for the full interpreter and backend and were subsequently executed by the accepted MVP. The separate SPI example has 32 executed decision scenarios and eleven consent histories, with its own recorded evidence.

F.2.2 The SPI-Demo1 input and output boundary

The public API accepts an immutable `Snapshot`, a time being assessed and a knowledge cutoff. A `Snapshot` binds a subject identifier to the declared fact map. A `Fact` carries a declared scalar type and one of `known`, `unknown` or `conflict`. A known Boolean is a Java `Boolean`; a known monetary or count value is a `BigInteger`. Its evidence identifiers, validity interval and recording time are mandatory. Unknown has a reason and no value. Conflict identifies competing evidence.

The host supplies every declared field, using an explicit unknown where evidence is unavailable. Omitting a field or supplying the wrong type is a malformed request, not a legal conclusion. Validity intervals are half-open: evidence valid until 10:00 does not establish a fact at 10:00. Evidence recorded after the knowledge cutoff is unavailable to that replay. The API takes canonical UTC timestamps with six fractional digits; conversion from a Hong Kong business timestamp belongs in the host adapter and must preserve the intended instant.

Listing F.1: The library API and one dated monetary input.

```
DecisionRuntime.Result evaluate(
    DecisionRuntime.Snapshot snapshot,
    String validAt,
    String knownAt);

// A known value is exact and supported by dated evidence.
Fact portfolio = Fact.known(
    "money_hkd", new BigInteger("4000000000"),
    List.of("valuation.lee", "ownership.lee"),
    "2026-09-01T00:00:00.000000Z",
    "2026-10-01T00:00:00.000000Z",
    "2026-09-01T00:00:00.000000Z");

Fact missingPortfolio = Fact.unknown("money_hkd", "MISSING");
```

The first amount is HK$40 million expressed in cents. A bank adapter reading a decimal amount should specify the currency, use exact decimal arithmetic and reject an unsupported fraction of a cent after the adopted conversion. In Java, `amount.movePointRight(2).toBigIntegerExact()` can enforce exact cents after that policy has been applied. Calling `doubleValue()` first would discard the property the specification is trying to preserve. For a currency conversion, the exchange-rate source, date, direction and rounding rule are part of the evidence mapping.

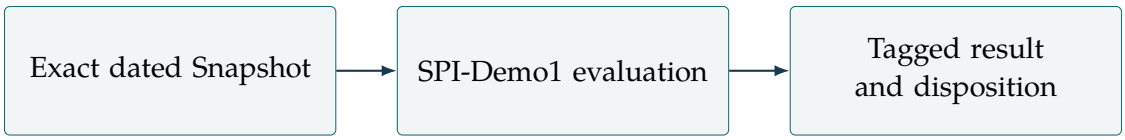


Figure F.2: The earlier Java interface illustrates the input and output boundary.

The result contains a logical status and an operational disposition:

Status	Disposition	Host meaning
`TRUE`	Conditions met	Selected streamlining conditions established; continue the other controls.
`FALSE`	Standard process required	At least one required condition is refuted; do not apply this streamlined route.
`UNKNOWN`	Review required	Evidence can change the conclusion; show blocking inputs.
`CONFLICT`	Review required	Resolve inconsistent evidence; preserve both records.
`OUT_OF_SCOPE`	Outside profile	Select another implemented profile or the standard process.

The demo's JSON includes `profile`, `authority`, both timestamps, the bundle and snapshot hashes, a postorder rule trace, missing and blocking input lists, and the result hash. A rule-trace entry gives its identifier, status, source-anchor identifiers and blocking inputs. Source anchors resolve through the bundle included with the release. Known values and their evidence remain in the preserved snapshot. The current RuleIR implementation provides expression-node traces, standardized diagnostics and engine identity through the different v0.1 API documented later in this chapter.

A malformed request throws an explicit Java input exception in this small library. The production adapter must map that exception, a timeout or an unavailable release to a technical-failure result and a review/standard-process path. It must never catch an exception and substitute a successful Boolean. The demonstration result always carries `DEMONSTRATION_ONLY`; production authority requires a different, approved release record and enforcement by the host deployment process.

F.2.3 How the compiler emits final Java rather than a suggestion

The emitter first validates the JSON schema, references, identifiers, types and acyclic rule graph. It then assigns an internal Java method name to each rule and recursively emits its expression. Identifiers are validated data, never pieces of executable text supplied by a language model. The generated monetary comparison is represented by actual Java calls such as:

Listing F.2: The generated financial method's essential body. The complete class is built and tested in the supplied example.

```
return c.rule("spi.financial", sourceIds, () ->
    any(
        compare("ge", c.fact("portfolio"),
            Value.of(new BigInteger("4000000000"))),
        compare("ge", c.fact("net_assets_ex_home"),
            Value.of(new BigInteger("8000000000")))
    ));
```

`compare` returns a known Boolean when both operands are known, or an unknown with its blocking input names. `any` implements the truth table in Equation (3.9). Java evaluates its arguments before calling the method, so both branches have the specified eager evaluation and evidence collection. Replacing this with Java's short-circuit `||` would change the trace and missing-input behavior even where the final Boolean agreed. The context memoizes referenced rules and records their results in a stable order. A precheck detects conflicting facts in this profile's dependency set before normal expression evaluation.

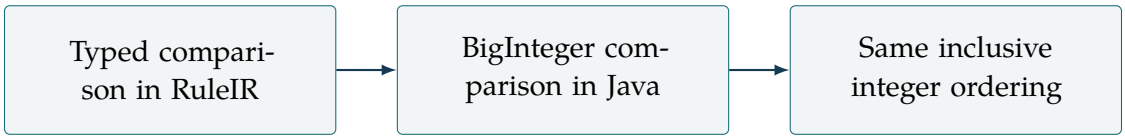


Figure F.3: The monetary comparison preserves meaning only if the input mapping preserves units.

The production emitter should implement one mapping per supported operator, with an explicit error for every unsupported construct. It must not ask an LLM to write new Java on each regulatory change. The LLM can propose the typed specification; after review, a deterministic compiler performs the mechanical translation. This concentrates semantic review on a small language and makes the generation step reproducible.

For the money comparison, the preservation argument is short. The IR represents both amounts as mathematical integers of HK cents. The Java boundary constructs the same integers as `BigInteger` values. The emitted comparison returns true exactly when `compareTo` is nonnegative, which expresses the same inclusive ordering. The argument depends on the input mapping preserving those integers. It would fail if the adapter truncated decimals, mixed currencies or overflowed a narrower integer before

constructing the value. For a complete compiler, analogous local arguments must be composed for every operator, unknown/error case, rule reference and trace requirement. The delivered tests check examples of preservation; they are not that universal proof.

F.2.4 Reproduce the build and see the host result

The repository retains a project-local Temurin JDK 17 for this check, verified against the official archive checksum. It does not change the machine's default Java installation. From the repository root, the single build command is:

```
python3 examples/java-dry-run/build_and_verify.py \
  --jdk .localresources/java-toolchain/jdk-17.0.20.1+1
```

The Python authoring check uses the already available JSON Schema package. The generated Java library itself has no third-party runtime dependencies. The build uses `javac --release 17 -Xlint:all -Werror`; creates the JAR; compiles the separate fixture and host callers against that JAR; compares complete reference and Java results; and runs the three mutations. The package has fixed ZIP timestamps to make repeated builds with the same toolchain reproducible. The verification record stores compiler identity, source/compiler/JAR digests, the executed command, results and limitations.

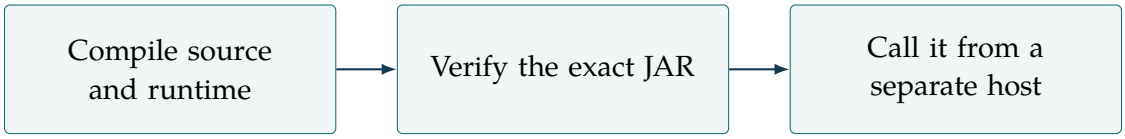


Figure F.4: The demonstration crosses the actual Java library boundary.

The delivered files below are operational copies of the design, not additional reading needed to understand why it works:

File under `examples/java-dry-run/`	Purpose
`spec/spi-control.bundle.json`	Source-linked typed interpretation for the chosen profile.
`spec/decision-cases.json`; `spec/consent-cases.json`	Independent expected statuses and synthetic evidence/history inputs.
`reference.py`; `compile_java.py`	Separate reference evaluation and deterministic Java generation.
`generated/GeneratedSpi.java`	Final generated decision class.
`src/main/java/hk/legalmath/spi/`	Evidence, result, three-valued arithmetic/logic support and consent replay.
`build/spi-controls-demo.jar`	Compiled library callable by a Java host.
`generated/BankHostExample.java`	Complete synthetic host caller using the public API.
`build/verification.json`; `build/host-output.txt`	Executed validation and host results.

The separate host prints these dispositions for the synthetic history:

```
BEFORE_WITHDRAWAL STREAMLINING_CONDITIONS_MET
AFTER_WITHDRAWAL STANDARD_PROCESS_REQUIRED
RESULT_AUTHORITY DEMONSTRATION_ONLY
```

This is the concrete answer to how final code reaches Java: the bank depends on the released library, constructs the declared evidence snapshot through its adapter, calls the generated class and handles its structured result. The compiler, public circulars and drafting tools are build-time/review-time components. They need not be deployed into the bank's order-processing process.

F.3 The actual generated Java interface

The current emitter validates the bundle, canonicalizes it and embeds its exact bytes in a generated class. The class name is `hk.legalmath.Policy_` followed by the first twenty hexadecimal digits of the bundle hash. The class exposes the full bundle hash and delegates evaluation to the Java semantic runtime. This is a generated, immutable policy binding; it is not the older proposal's imagined record-based API or a proof that each legal rule has been translated to hand-optimized Java control flow.

The two current entry points are:

Listing F.3: Implemented generated-policy API.

```
public static String evaluate(
    String snapshotJson,
    String ruleId,
    String validAt,
    String knownAt,
    String mode);

public static Map<String,Object> evaluate(
    Map<String,Object> snapshot,
    String ruleId,
    String validAt,
    String knownAt,
    String mode);
```

The string overload returns strict JSON. The map overload returns a deeply immutable result. The generated policy and Java runtime operate without Python, a language model, a solver or network retrieval. This property is important for a bank integration: the interpretation workbench prepares and verifies a package, while production decision execution can remain local and deterministic.

A concrete host compiles against the actual generated class name from the build manifest. It supplies a validated snapshot, exact rule identifier, both assessment times and the requested mode. The mode does not grant authority. A caller cannot turn a draft

into an approved control by passing the word “production.” The release service and host must establish approval and applicability separately.

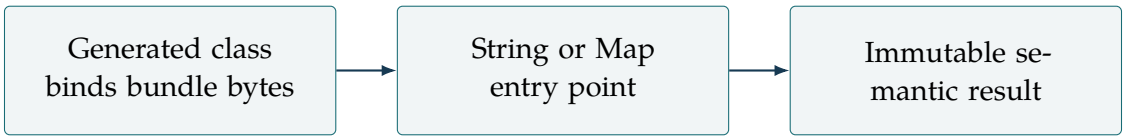


Figure F.5: The current generated policy delegates to the verified runtime.

The conceptual integration looks as follows. The symbolic class name in the listing is replaced by the actual generated name when the integration fixture is built; the release lookup and transaction methods are host responsibilities.

Listing F.4: Host responsibilities around deterministic evaluation.

```
ApprovedRelease release = releases.resolve(profile, validAt, knownAt);
release.verifyAuthorityAndDigests();
ConsistentSnapshot s = adapter.readSnapshotWithVersions(subject, order);
String resultJson = Policy_HASH.evaluate(
    s.strictJson(), release.ruleId(), validAt, knownAt, "production");
Decision decision = decoder.decodeStrictly(resultJson);
HostDisposition disposition = hostPolicy.map(decision);
transactions.commitIfVersionsUnchanged(
    s.versions(), order.id(), order.payloadHash(),
    release.identity(), decision, disposition);
```

`ApprovedRelease`, `ConsistentSnapshot` and the host transaction interfaces in this illustration are proposed integration concepts, not classes already supplied by the generated SDK. Their responsibilities must be implemented in the bank’s environment. The example makes the boundary explicit so an agent does not mistake illustrative types for an existing dependency.

The host must handle every tagged result. A true selected prohibition condition may require stopping the relevant offer; false permits only progression to other controls under the host policy. Unknown, conflict, out-of-scope and error each have explicit dispositions. No result string should be cast to a Boolean by a language’s truthiness rules. The decoder rejects unknown status tags so that a future runtime extension cannot silently take an old default branch.

F.4 What can be reused from the adjacent projects

DynareMCP contains useful patterns for explicit hypotheses, diagnostics, falsifiers and limits of claims. In the inspected legacy audit writer, `write_hypothesis_node` records a hypothesis, source support, code support, a diagnostic, a falsifier and a supplied rank score. These fields map naturally to interpretation candidates. The supplied score is not automatically calibrated; its meaning depends on the caller.

The inspected legacy scheduler chooses among eligible frontier nodes by a stored priority and identifier. It is not a general UCT engine hidden behind the word "tree." Reusing its bookkeeping ideas is sensible; importing it and claiming that a proven Monte Carlo search now verifies legal meaning would be wrong. The new controller needs its own issue identity, source lineage, legal review states and resource accounting.

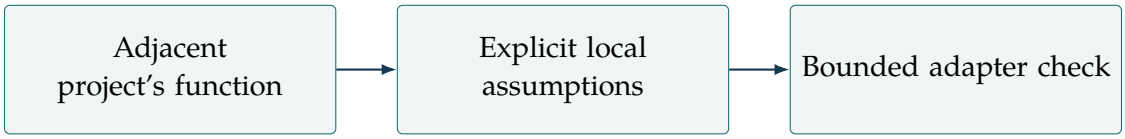


Figure F.6: Reuse must name a concrete capability and the changed context.

MathDevMCP is useful once a proposition is precise enough to prove. A formal obligation might concern preservation of a Boolean expression, absence of an unbounded loop in a finite controller model, or equivalence of two encodings over a declared domain. ResearchAssistant supports retrieval, parsing and retention of the literature and regulatory evidence. Neither project supplies a privileged interpretation of English. They strengthen different links in the chain.

The design that follows therefore combines several structures: a source inventory to detect missing material, a candidate set to expose choices, an argument graph to record reasons, a bounded search to investigate questions, and formal checks to compare executable consequences. The diversity is useful because each structure can reveal mistakes that the others overlook. Its value will depend on how discrepancies move between them, which is the central subject of the next chapter.

F.4.1 Adapter boundaries and the earlier reuse assessment

The earlier repository inspection contributes the adapter requirements below. The preceding analysis identifies the actual search-controller differences; references here to historical UCB-style investigations do not override that finding or establish that a generic MCTS service is already available.

The adjacent projects supply reusable patterns and potential adapters. They are not interchangeable components of a legal reasoner. Their local code and documentation were inspected at the revisions recorded in the source manifest (LegalMath research notes, 2026b). The proposal makes no change to those repositories and does not assume that their current interfaces implement the proposed legal semantics.

ResearchAssistant is useful immediately for source intake, local storage, parsing, discovery and structured review records. It was used in this investigation to discover literature and parse retained PDFs. The available extraction route used `pdftotext` and reported low confidence requiring manual review. LegalMath needs a regulatory layer above it: circular identifiers, annex relationships, paragraph anchors, versions, replacement links and an applicability inventory. PDF extraction quality must be assessed on the actual source layout, especially footnotes and tables.

MathDevMCP is useful after a property has been formalized. Its inspected Lean route binds checks to an exact target and assumptions, and its result handling distinguishes diagnostic evidence from proof. Its source-bound obligations and resumable evidence structures can support the question in Equation (3.2). The legal adapter must define the rule language's semantics, numeric and date operations, and the exact theorem requested. Structural agreement between a formula and a code fragment is not a substitute for semantic equivalence. The inspected project describes an internal research status and has its own licensing terms; reuse within a bank needs the appropriate software and ownership review.

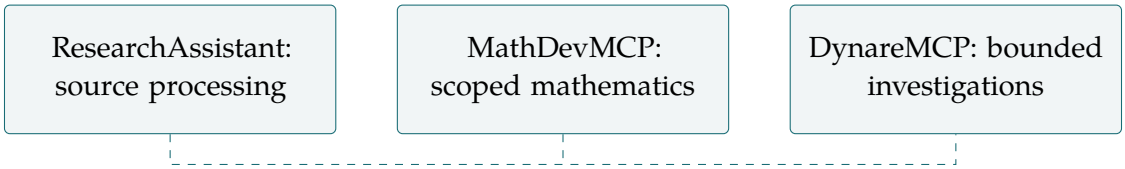


Figure F.7: Adjacent tools contribute different capabilities, each requiring local validation.

DynareMCP supplies ideas for investigative planning, source graphs and meaningful differences between model versions. Its economic-model parser and numerical solvers are not direct engines for SFC requirements. The reusable pattern is a reviewable change supported by a dependency graph, with unresolved alternatives preserved. A legal adapter would compare rules, definitions, scope and dates rather than macroeconomic equations.

The proposed search use is equally specific. Historical DynareMCP investigation records use UCB-style exploration to choose work; the currently inspected MathDevMCP derivation controller explicitly describes a different bounded evidence-ordering method, not MCTS. It would be inaccurate to label both as the same proof-search engine (LegalMath research notes, 2026b).

For LegalMath, a search node could contain a candidate interpretation and its unresolved questions. Actions might retrieve a referenced FAQ, request a counterexample, check a definition, or prepare a question for a compliance owner. A search score may prioritize expected information gain or reduction in review work. It cannot determine which interpretation is legally correct. A branch with more permissive outcomes or more successful solver calls receives no special authority. If several readings remain viable, the product retains the disagreement.

Start with a bounded work queue ordered by materiality and dependencies. MCTS or UCB scheduling becomes a candidate only if the prototype reveals a substantial search-allocation bottleneck. Its evaluation would compare useful questions resolved per unit of review effort under the same budget, while correctness continues to depend on source review and formal evidence. Transaction execution uses approved rules, with no exploratory search in the decision path.

F.4.2 A reproducible research collection

The papers are stored under `docs/papers/` using the requested filename convention: the title, an underscore, the first author's surname and the year in parentheses. Filesystem-unsafe title punctuation is normalized, while the manifest retains the original title. Each record includes its source URL, bibliographic identity, edition note, retrieval date, byte count, PDF page count and SHA-256 digest. The library index distinguishes distinct works from alternate editions and identifies discovery leads that were not available as usable full text.

The technical notes identify which part of each paper supports a proposed borrowing and what remains untested. They distinguish a semantics from its implementation, a reported experiment from a reproduced result, and a publicly available repository from a licensed and approved bank dependency. Full-text extraction is retained for local search, but the source PDF remains the reference where layout or equations matter. ResearchAssistant extraction is an aid to inspection, not certification of the extracted text.

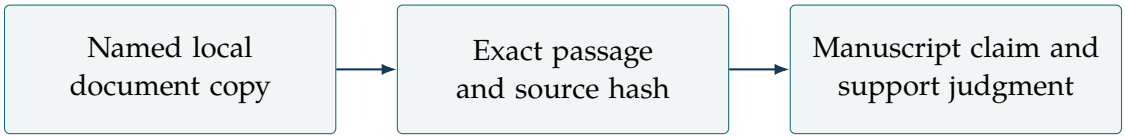


Figure F.8: An audit trail must show what was actually read for a citation.

The SFC source snapshots and the earlier local-repository inspection records remain under `.localresources/`, with their own manifest. The LaTeX and bibliography are now canonical in `docs/monograph/`; the earlier proposal entry point builds the same unified work. This arrangement permits another reviewer to trace the proposal's substantive examples back to a document edition without reconstructing the web searches. Redistributing the paper library is a separate matter from using the local copies for this research; the source licenses and access terms remain relevant.

F.5 Exact controller sequence

Listing F.5: Main controller, with explicit terminal uncertainty.

```
start_run(source_packet, profile, required_policy):
    validate_source_identity_and_policy()
    commit(run = INITIALIZING, counters = 0)
    reserve_and_issue_initial_roles_with_global_budget()
    collect_until_complete_or_action_deadline()
    validate_outputs_and_freeze_initial_proposals()
    reconcile_inventory_candidates_and_required_roles()

    while has_unresolved_issue():
        if integrity_failure():
```

```
            close_as(FAILED_INTEGRITY); break
        if cancelled():
            close_as(CANCELLED); break
        if deadline_or_global_or_round_limit_reached():
            mark_remaining_issues_with_stop_reason()
            close_as(BLOCKED_UNRESOLVED); break
        if no_progress_limit_reached():
            mark_remaining_issues(NO_PROGRESS)
            close_as(BLOCKED_UNRESOLVED); break

        plan = choose_permitted_discriminating_actions()
        if plan.is_empty():
            mark_remaining_issues(HUMAN_REQUIRED)
            close_as(BLOCKED_UNRESOLVED); break
        commit_next_round_and_reserve_bounded_actions(plan)
        dispatch_only_reserved_actions()
        collect_results_until_each_action_is_terminal()
        validate_results_and_create_candidate_revisions()
        rerun_affected_checks_and_reconcile_all_open_issues()
        record_progress_signature()

    if not already_closed():
        apply_integrity_cancellation_and_deadline_guards()
    if not already_closed():
        check_required_roles_coverage_and_all_mandatory_checks()
        if any_requirement_incomplete():
            close_as(BLOCKED_UNRESOLVED)
        else:
            close_as(READY_FOR_REVIEW)
    write_complete_report_for_every_terminal_status()
    return report_reference
```

F.6 Machine-readable blocked-result example

Listing F.6: A blocked result preserves alternatives and the next decision.

```
{
  "schema_version": "interpretation.v1",
  "run_status": "BLOCKED_UNRESOLVED",
  "source_ref": "23EC46-retained-version",
  "selected_slice": "paragraph-10",
  "candidate_ids": ["cashback-reimbursement", "cashback-fee-reduction"],
  "material_unresolved_issue_ids": ["classification-cashback"],
  "release_eligible": false,
  "processing_stop": "ROUND_LIMIT",
  "rounds_issued": 3,
```

```
  "probability_of_legal_correctness": null,
  "next_decision": "Establish the applicable fee-discount definition",
  "next_decision_owner_role": "LEGAL_INTERPRETATION_REVIEWER"
}
```

F.7 Exact financial-rule serialization

The complete financial-condition bundle is supplied in `fixtures/spi-financial.bundle.json` beneath the specification directory. It declares two money inputs, an unconditional scope for this isolated subcondition, the expression, interpretation record and a verified span in Annex 1 paragraph 3.1. The excerpt below is a complete expression node from that bundle. Node identifiers make its trace reproducible.

Listing F.7: The portfolio comparison in the supplied JSON bundle. Amounts are HK cents.

```
{"node_id":"portfolio.ge", "op":"compare", "cmp":"ge",
 "left":{"node_id":"portfolio.input", "op":"fact",
         "name":"portfolio"},
 "right":{"node_id":"portfolio.threshold", "op":"literal",
          "type":"money_hkd", "value":"4000000000"}}
```

The second comparison uses `net_assets_ex_home` and `"8000000000"`; an `any` node combines them. These inputs denote amounts normalized under adopted wealth definitions, not any account balance supplied under the same name. Release requires those definitions and applicability to be reviewed. The narrow result name, `spi.financial`, identifies a subcondition rather than a complete SPI assessment or trade authorization.

F.8 Complete rule-language grammar

An implementer can express the grammar below directly as tagged records. Every `Expr` additionally carries a unique `node_id`. A rule is `(id, type, scope, body, interpretation_id, source_span_ids)`; `scope` is Boolean and `body` has the declared result type.

Listing F.8: Complete operator grammar for RuleIR 0.1; repeated arguments are ordered. The Java demonstration implements a stated subset.

```
Type = bool | integer | money_hkd | date
Expr = literal(Type, value)
     | fact(name) | rule(name)
     | all([Expr, ...]) | any([Expr, ...]) | not(Expr)
     | compare(eq | ge | gt, left, right)
     | add(left, right) | sub(left, right)
     | scale(arg, numerator, denominator)
```

```
     | if(condition, then_value, else_value)
     | default(base, [Exception, ...])
Exception = (exception_id, guard, value,
             interpretation_id, source_span_ids)
```

F.9 Durable request accounting and worker recovery

An action moves through `RESERVED`, `DISPATCHED`, and one of `SUCCEEDED`, `FAILED`, `TIMED_OUT`, or `RESULT_UNKNOWN`. The last state is necessary when the controller cannot determine whether a remote service performed the action. A lost connection after submission is different from a request rejected before dispatch. Both can lead to missing results, but only the former may already have consumed remote work.

The reservation transaction records the action identifier, input hashes, issue root, round, budget charge, adapter identity, timeout and idempotency key. An outbox entry is written in the same transaction. A worker claims that entry with a lease and a fencing token. The token increases when ownership changes, so an old worker cannot commit a result after a newer owner has taken responsibility. A valid late result can be retained as an observation without changing the closed action's authoritative state.

The dispatch boundary requires care. If the outbox records only "not yet sent" and a worker crashes after sending but before updating the database, another worker may send again. A provider idempotency key can make that retry refer to the same operation, but not every provider supports this guarantee. The adapter must state its capability. Without remote idempotency, a recovery policy can choose to leave the result unknown rather than risk another paid request, or authorize a new counted action. It cannot truthfully promise exactly once remote execution.

The budget system reserves an upper bound on tokens or cost when those resources are capped. Settlement records actual usage when known. A refund of unused monetary reservation may be legitimate after a confirmed terminal response, but it does not decrement the issued-action count. Otherwise a sequence of failed requests could evade the maximum number of attempts. If actual usage exceeds a provider's documented bound, the run records an accounting discrepancy and stops new spending until the integration is investigated.

A compare-and-swap version protects issue updates. The worker returns evidence against the issue revision it received. If the issue has changed, the controller revalidates whether the evidence is still relevant before applying a resolution. This prevents two simultaneous actions from overwriting each other's objections or resolving an issue using a candidate that is no longer current.

Cancellation stops new reservations and attempts to cancel outstanding work. Remote cancellation may be best effort. The report records outstanding or late actions and keeps their budget charges. Cancellation cannot convert an incomplete run into a clean result, and it cannot delete the source of a later adverse finding. These details matter because regulatory evidence must survive ordinary operational events, including a person clos-

ing a browser or restarting a worker.

F.9.1 Scope and method of this review

The review began with the seven requested links and expanded through their references, forward-citation searches for Catala and Stipula, and searches on executable law, legal reasoning, formalization, dates, testing and process compliance. ResearchAssistant supplied discovery records and local PDF extraction. The saved OpenAlex citation searches returned 52 entries for the selected Catala record and 13 for the selected Stipula journal record. Those are retrieval counts for particular indexed editions, not complete or comparable measures of influence. Preprints, revised editions and new publications can be missing or duplicated.

The initial proposal retained 38 paper editions representing 37 distinct works. The first unified library retained 50 editions representing 49 works, including the later legal-search and uncertainty studies. The present audit adds two legal-reliability studies, bringing the academic collection to 52 editions representing 51 works. Regulatory documents, standards and software records are retained alongside them. The AAAI tax-reasoning paper and its extended preprint are retained separately; all seven requested sources are present. Technical reading covered the relevant methods, semantics, evaluation and supporting appendix material. The accompanying notes identify the inspected sections, version differences, implementation checks and limits of adoption (LegalMath research notes, 2026a). This is a focused technical review supporting a product decision, rather than a claim to have enumerated every citation. Unavailable secondary leads are recorded separately and do not support implementation claims.

The expanded search treats eight families separately: executable statutes; defaults and defeasible priorities; obligations and violations; temporal process compliance; authority and provenance; legal languages, compilers and testing; neurosymbolic formalization and evaluation; and deployable rule engines. Important foundations predate Catala, while contract languages have partially separate citation communities. The retained forward-citation records have individual dispositions in `docs/papers/coverage.md`; additional searches and backward references supply the missing mechanisms. Selection favors primary sources that define a needed mechanism or report an informative failure. Downloading a paper or screening its abstract is distinguished from inspecting its technical method. This is a bounded review of the prototype's design questions. Remaining retrieval gaps are stated rather than counted as read papers.

Version control is particularly important here. The ARc paper was first posted in 2025, but the inspected second version is from July 2026. The eFLINT paper's inspected version is from August 2026. The Stipula preprint requested by the sponsor has three authors; the later journal edition has a different author list. Bibliographic records and local filenames make these distinctions explicit. Published results are reported as their authors' findings; none of their benchmarks or proof developments was independently rerun for this proposal.

F.10 Release identity and reproducible builds

A build manifest records the bundle, generated source, runtime source, toolchain, Java target, dependencies and JAR digest. A verification report then names the build and exact JAR it tested. A release manifest names that build and verification report, together with applicability and effective-time information. Approval binds the final release manifest and bundle.

The direction matters. Embedding a manifest that contains the JAR's own digest inside that JAR creates a circular identity problem. The current design keeps these records external and acyclic. A reviewer approves the exact tested bytes through the final manifest, rather than approving a mutable directory or a human-readable version label.

Candidate compilation occurs before legal approval, because reviewers need a concrete package and verification evidence to inspect. Compilation does not grant deployment authority. Export into a controlled release route occurs only after the required approvals and checks. This distinction lets engineers do useful reversible work without repeatedly asking for permission, while preserving the human decision at the point where it matters.

Reproducible builds strengthen the identity chain. Under the same declared toolchain and inputs, build in separate directories with fixed archive-entry timestamps and compare emitted source and binary hashes. A mismatch should be investigated and recorded. Reproducibility does not prove semantic correctness, but it reduces uncertainty about which bytes correspond to the reviewed source.

The local MVP uses synthetic identities and unsigned historical archives to exercise workflow. A bank deployment needs enterprise identity, role separation, key management and independently verified release signatures or equivalent controls. A hash proves equality of bytes under its cryptographic assumption; it does not prove who authorized those bytes. The manuscript's implementation plan keeps that integration distinct from the local demonstration.

Bibliography

Accord Project. Cicero, 2026. URL https://github.com/accordproject/cicero. Official repository documentation and stated licensing inspected September 2026.

Apache KIE contributors. Drools, 2026. URL https://github.com/apache/incubator-kie-drools. Official repository documentation inspected September 2026.

Joaquín Arias, Mar Moreno-Rebato, José A. Rodríguez-García, and Sascha Ossowski. Automated legal reasoning with discretion to act using s(LAW). Artificial Intelligence and Law, 2024. URL https://arxiv.org/pdf/2401.14511. 2024 arXiv copy; journal first published online in 2023.

Catala contributors. Catala: compiler and core formalization repository, 2026. URL https://github.com/CatalaLang/catala. README, formalization notes and selected implementation/proof source inspected September 2026; proof not replayed.

Guido Governatori, Mustafa Hashmi, Ho-Pun Lam, Serena Villata, and Monica Palmirani. Semantic Business Process Regulatory Compliance Checking Using LegalRuleML. EKAW 2016, 746–761, 2016. URL https://inria.hal.science/hal-01572441/document. HAL author manuscript.

Legalese contributors. L4 compiler, IDE and DEONTIC reference, 2026. URL https://github.com/legalese/l4-ide. README and doc/reference/regulative/DEONTIC.md inspected; no execution or equivalence claim.

LegalMath research notes. Paper library, technical reading notes and retrieval manifest. Local files: docs/papers/README.md, reading-notes.md and manifest.json, 2026a. 38 PDF editions representing 37 works; expanded notes and coverage ledger; research snapshot 21 September 2026.

LegalMath research notes. Local project capability and source review. Local files: docs/sfc-rule-translator-design.md and .localresources/source-manifest.json, 2026b. MathDevMCP 655a9efeb338, DynareMCP 4bac9dd9733c, ResearchAssistant 9c0008f5c41b; inspected September 2026.

Lexpedite. Blawx, 2026. URL https://github.com/Lexpedite/blawx. README describes educational and experimental use; inspected September 2026.

OASIS LegalRuleML Technical Committee. LegalRuleML Core Specification Version 1.0, 2021. URL https://docs.oasis-open.org/legalruleml/legalruleml-core-spec/v1.0/os/legalruleml-core-spec-v1.0-os.html. OASIS Standard, 30 August 2021.

Object Management Group. Decision Model and Notation, Version 1.5, 2024. URL https://www.omg.org/spec/DMN/1.5/PDF. Formal specification; hit-policy discussion in section 8.2.11.

Open Policy Agent contributors. Policy Language: Rego, 2026. URL https://www.openpolicyagent.org/docs/policy-language. Official documentation inspected September 2026.

OpenFisca contributors. OpenFisca Core, 2026. URL https://github.com/openfisca/openfisca-core. Official repository documentation inspected September 2026.

Sepehr Sharifi, Alireza Parvizimosaed, Daniel Amyot, Luigi Logrippo, and John Mylopoulos. Symboleo: Towards a Specification Language for Legal Contracts. IEEE International Requirements Engineering Conference 2020, 2020. URL https://drive.google.com/uc?export=download&id=1WXwXeLrZdaJjhSJcCrt_wBXxDvhFkq2k. Author preprint linked by official Symboleo repository; some ancillary semantics references remain anonymized.

Smart Contract Modelling, University of Ottawa. Symboleo editor and model checker, 2026. URL https://github.com/Smart-Contract-Modelling-uOttawa/Symboleo-Model-Checker. Official editor README, sale example and model-checker README inspected; no nuXmv replay.

Stipula contributors. Stipula implementation and workbench, 2026a. URL https://github.com/stipula-language/stipula-workbench. Official project repositories; workbench and prototype documentation inspected September 2026.

Stipula contributors. Stipula-to-Java/JML verification implementation, 2026b. URL https://github.com/stipula-language/stipula-KeY. Official translation repository; Translator.java and documentation inspected, no KeY replay.

L. Thomas van Binsbergen, Christopher A. Esterhuyse, and Tim Müller. Reflections on the design, applications and implementations of the normative specification language eFLINT. Journal of Computer Languages, 2026. URL https://arxiv.org/pdf/2511.12276v3. Reviewed arXiv v3, 4 August 2026; journal title differs slightly; first preprint 2025.

Z3 contributors. Z3 theorem prover, 2026. URL https://github.com/Z3Prover/z3. Official repository documentation inspected September 2026.